厦门大学信息学院 本科选修课
2024-2025 第二学期

模 式 识 别
Pattern Recognition

主 讲：王程教授、陈龙彪副教授
厦门大学信息学院计算机科学与技术系
人工智能引论101计划系列课程



本节概览

01 卷积结构的类型

02 常用的卷积神经网络

03 模型压缩与加速概述

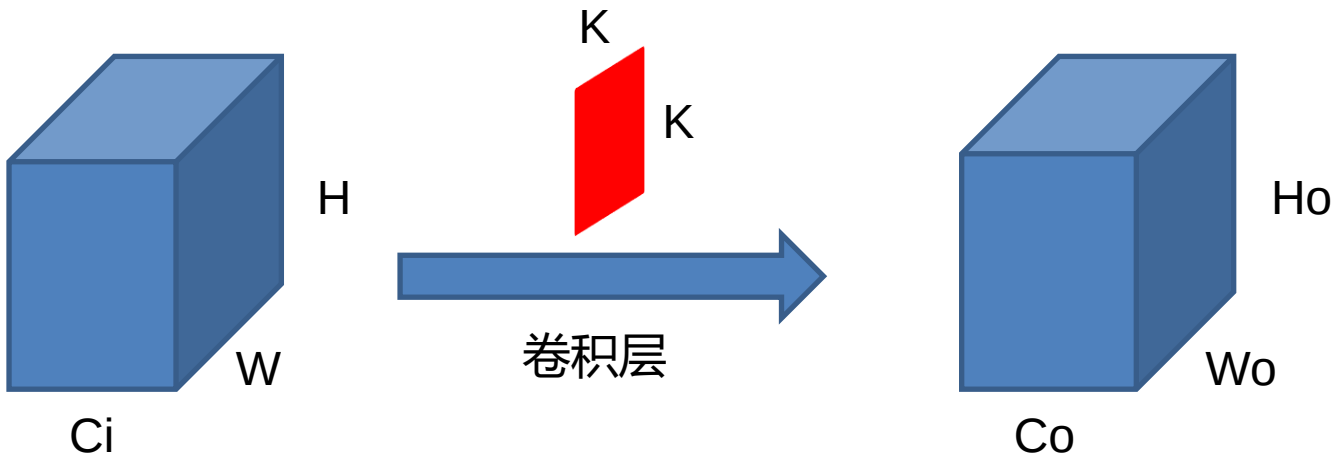
04 模型压缩与加速方法

第 1 部分

卷积结构的类型

卷积结构的类型

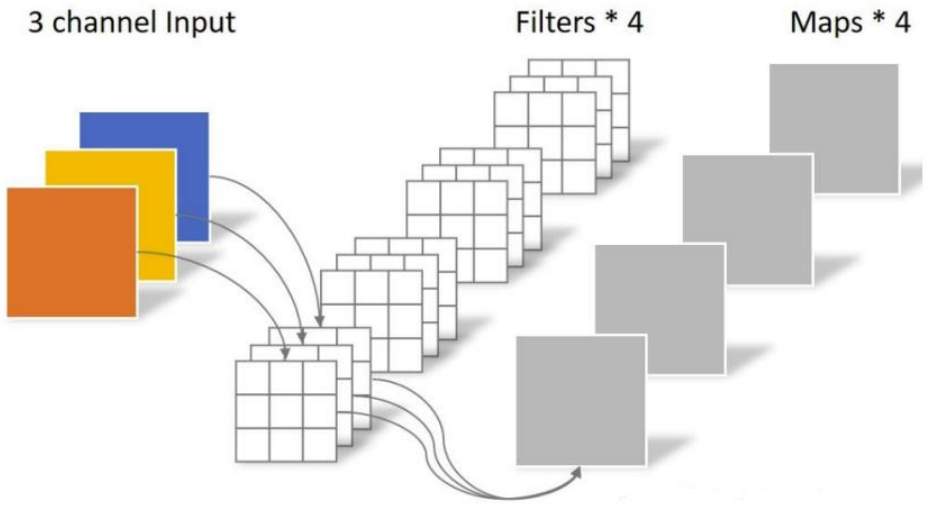
正常卷积(Convolution)



Ho=(H+2×padding+1-K)/stride

Wo=(W+2×padding+1-K)/stride

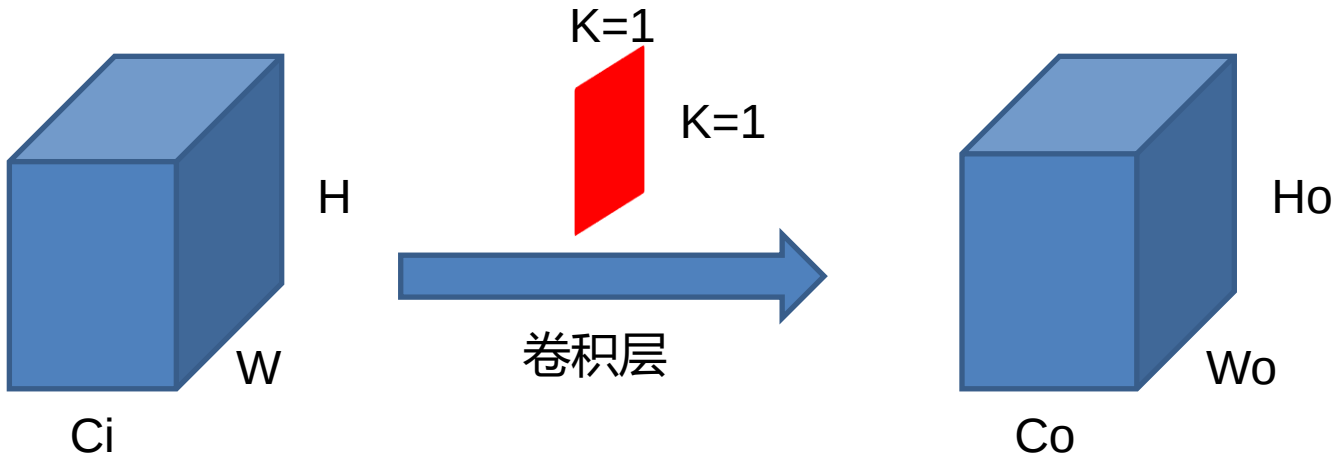
参数量：Ci×K×K×Co+bias



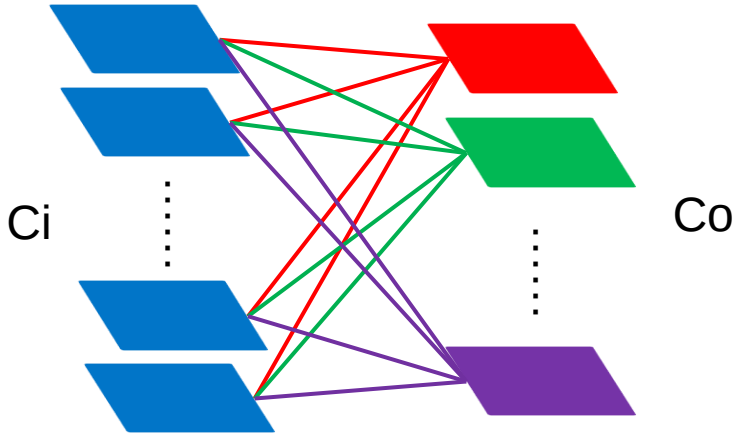
torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*)

卷积结构的类型

逐点卷积(Pointwise Convolution)



- kernel_size = 1
- 参数量：Ci×Co+bias
- 该卷积操作没有空间信息
- 通道维度上的全连接

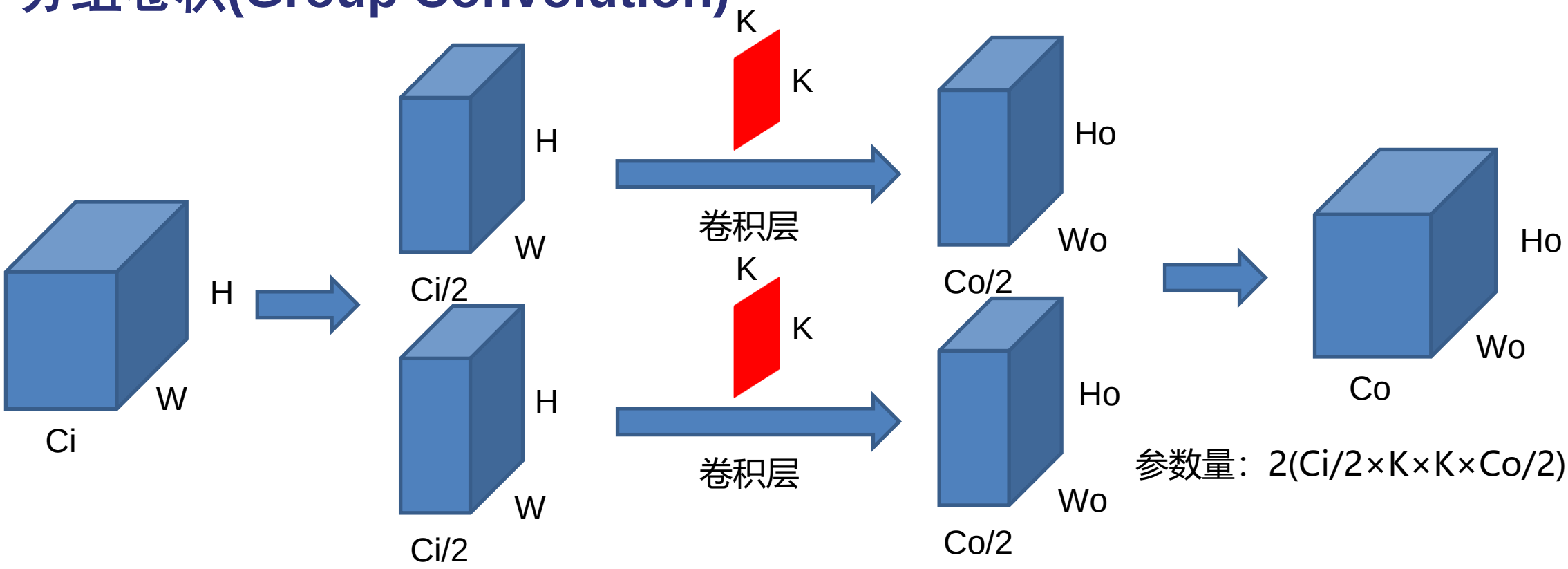


1×1卷积示例

torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*)

卷积结构的类型

分组卷积(Group Convolution)



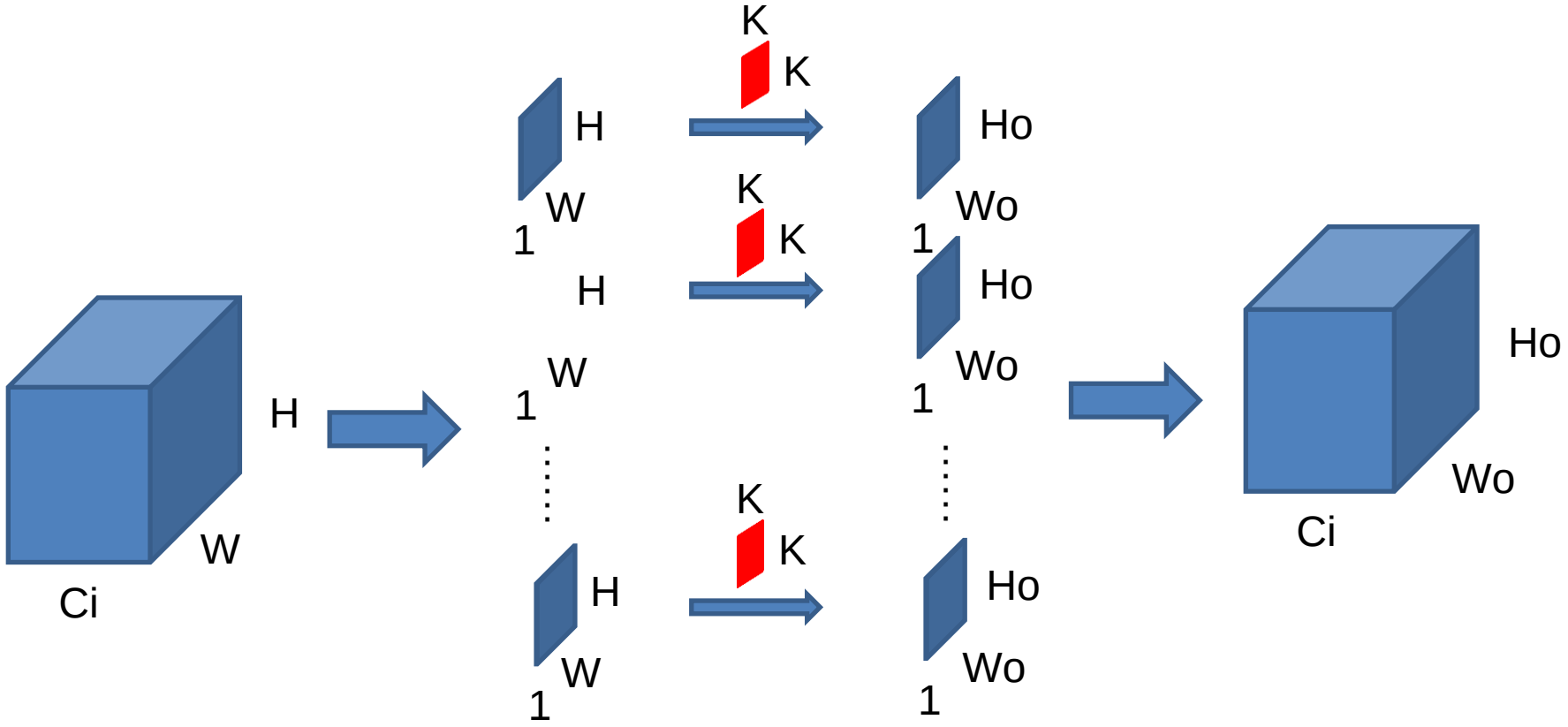
参数量：2(Ci/2×K×K×Co/2)+bias

torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, ***groups=2***, *bias=True*)

- **groups** (*int*, *optional*) – Number of blocked connections from input channels to output channels. Default: 1

卷积结构的类型

深度卷积(Depthwise/Channel-wise Convolution)



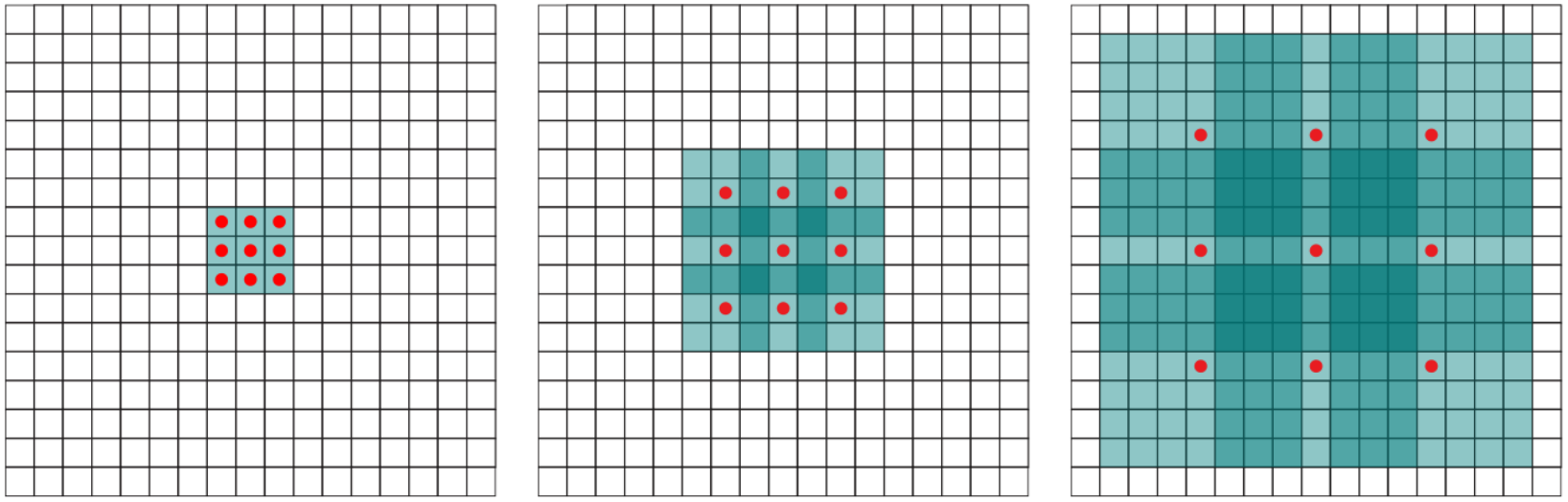
- 参数量：Ci×K×K+bias
- 分组卷积的极端形式
- 参数量少
- 通道之间的信息没有打通
- 通常后接1×1卷积打通通道信息

torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, ***groups=Ci***, *bias=True*)

- **groups** (*int*, *optional*) – Number of blocked connections from input channels to output channels. Default: 1

卷积结构的类型

空洞卷积(Dilated Convolution)



(a) (b) (c)

- 参数量：Ci×K×K×Co+bias
- 参数量不变
- 扩大感受野，在分割任务中常见
- 提取多尺度的特征

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

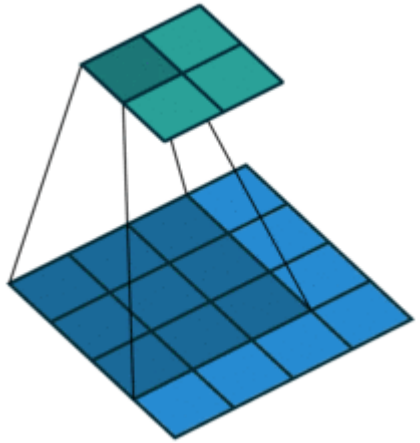
$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

torch.nn.Conv2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=2*, *groups=2*, *bias=True*)

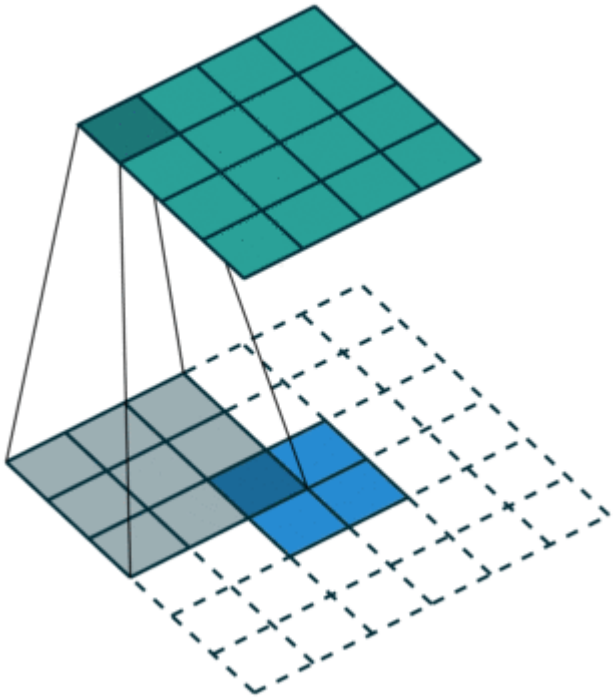
- **dilation** (*int* or *tuple*, *optional*) – Spacing between kernel elements. Default: 1

卷积结构的类型

转置卷积/反卷积(Transpose Convolution)



反卷积



卷积

- 参数量：Ci×K×K×Co+bias
- 卷积的逆操作
- 可学习的上采样层，在图像分割，图像生成中广泛应用

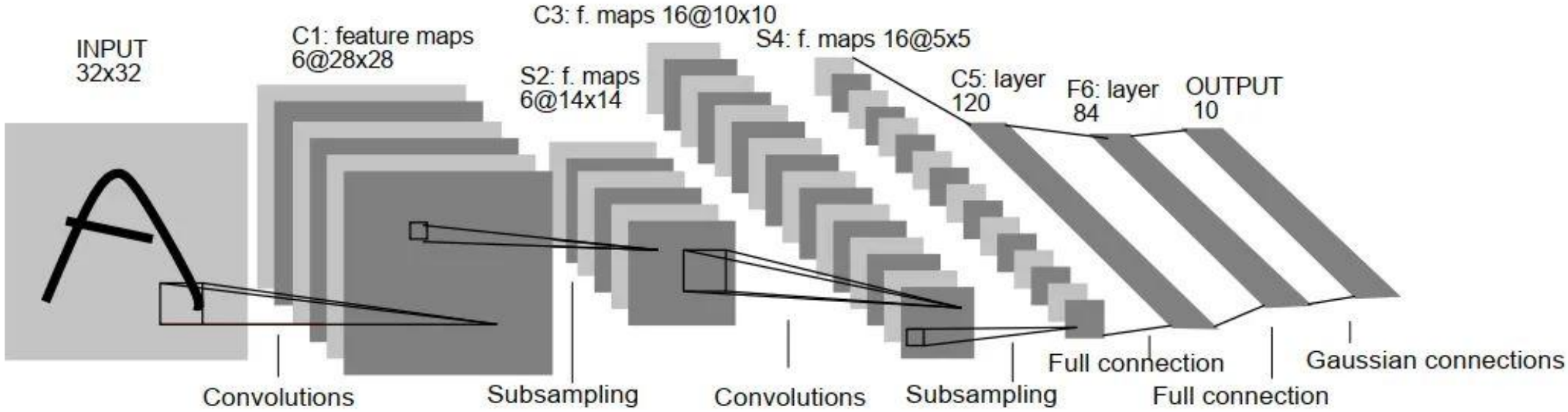
torch.nn.ConvTranspose2d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *output_padding=0*, *groups=1*, *bias=True*, *dilation=1*)

第 2 部分

常用的卷积神经网络

常用的卷积神经网络

LeNet-5 (1990's)



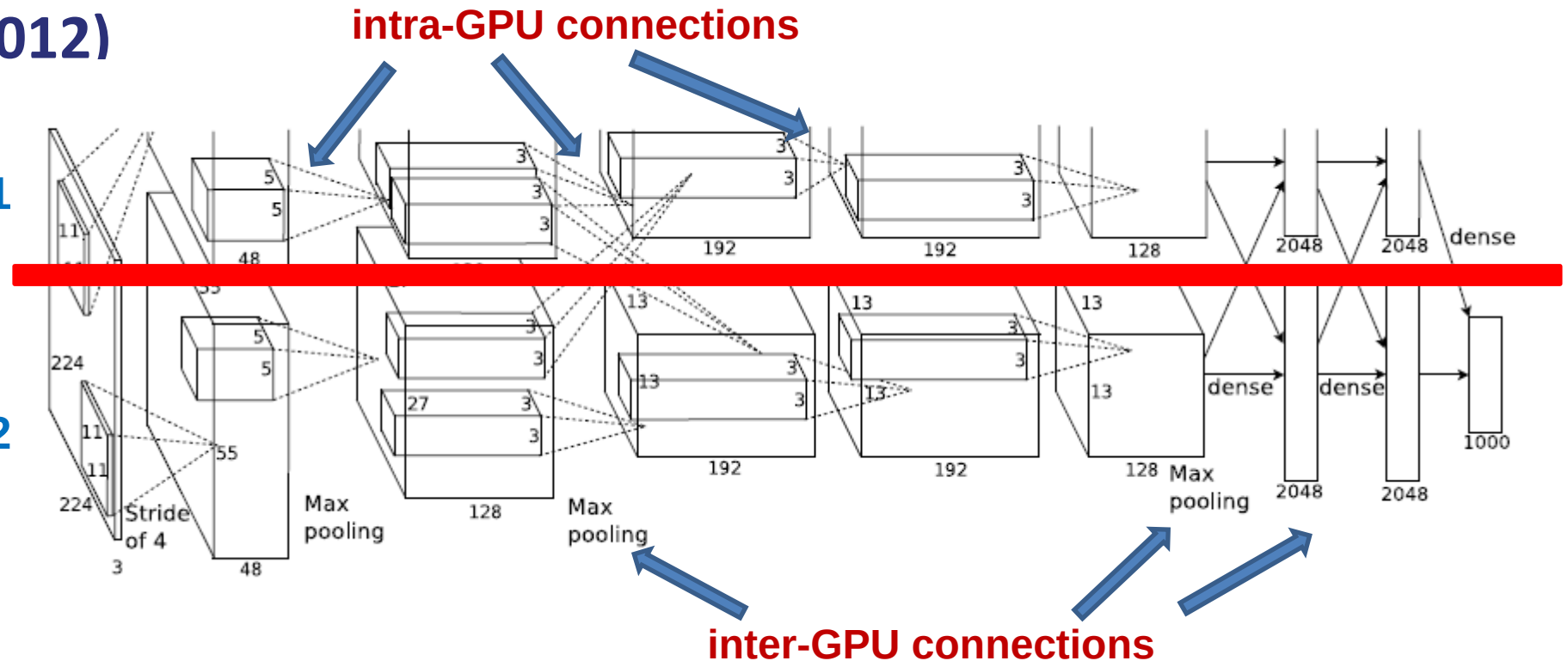

```
class LeNet5(nn.Module):
    def __init__(self):
        super(LeNet5).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, padding=2)
        self.conv2 = nn.Conv2d(6, 16, 5)

        self.fc1 = nn.Linear(16*5*5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
def forward(self, x):
    x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2))
    x = F.max_pool2d(F.relu(self.conv2(x)), (2, 2))
    x = x.view(-1, self.num_flat_features(x))
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x
```

常用的卷积神经网络

AlexNet (2012)

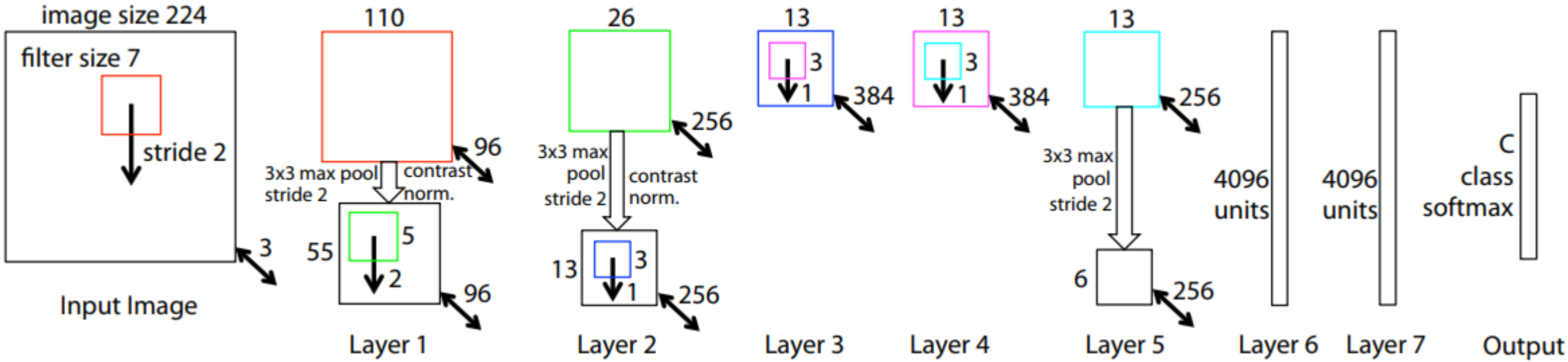


AlexNet有5个卷积层和3个全连接层，移除任意一层都会降低最终的效果

- Multiple GPU
- Group convolution
- ReLU
- Max pooling
- Dropout
- Local normalization

常用的卷积神经网络

ZFNet (2013)



ZFNet在保留AlexNet的基本结构的同时利用反卷积网络可视化的技术对特定卷积层的卷积核尺寸进行了调整，第一层的卷积核从11*11减小到7*7，将stride从4减小到2，Top5的错误率比AlexNet比降低了1.7%。

常用的卷积神经网络

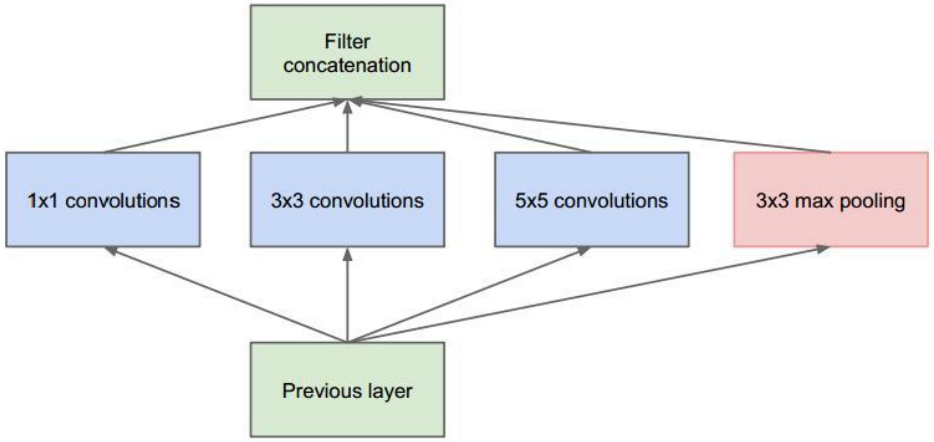
VGGNet (2014)



- 更深的网络
- 小卷积核的堆叠
- 卷积Block的重复
- 通道数呈现二进制的增加

常用的卷积神经网络

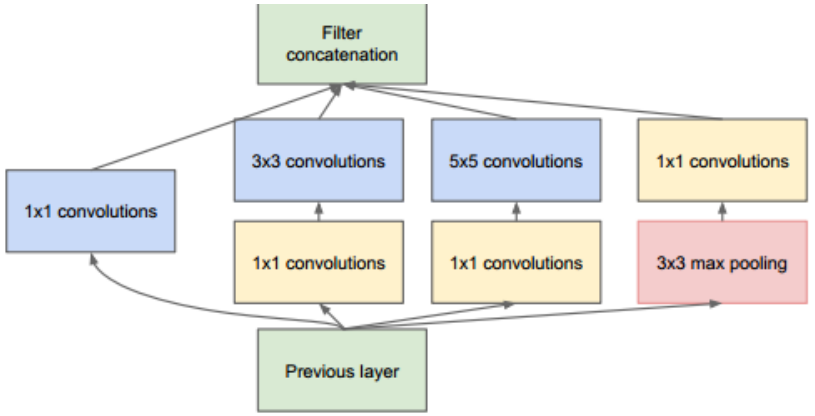
GoogLeNet (2014)



(a) Inception module, naïve version

Inception-v1

- 不同尺度的feature map的融合



(b) Inception module with dimension reductions

Inception-v2

- Channel-wise convolution减少参数

常用的卷积神经网络

GoogLeNet (2014)

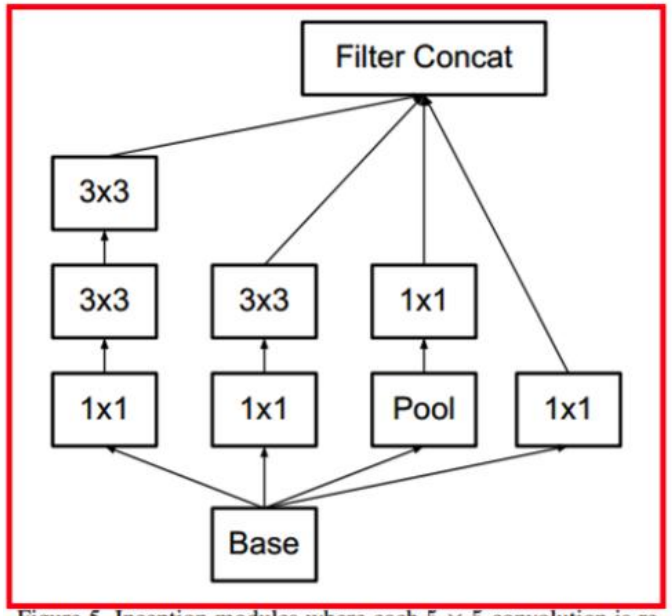


Figure 5. Inception modules where each 5×5 convolution is replaced by two 3×3 convolution, as suggested by principle 3 of Section 2.

Inception-v2

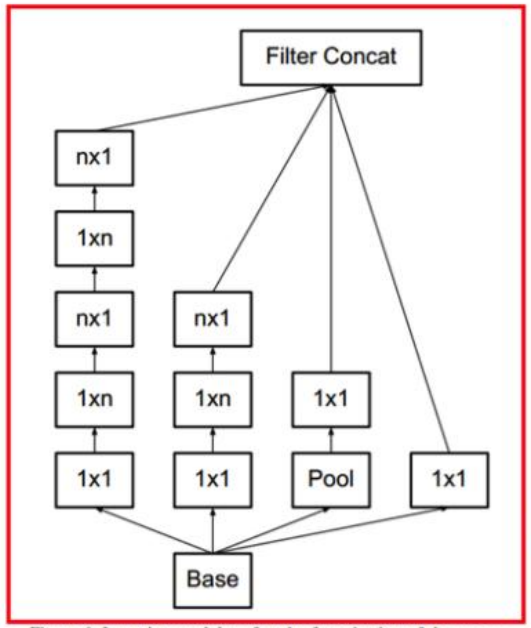


Figure 6. Inception modules after the factorization of the $n \times n$ convolutions. In our proposed architecture, we chose $n = 7$ for the 17×17 grid. (The filter sizes are picked using principle 3)

Inception-v3

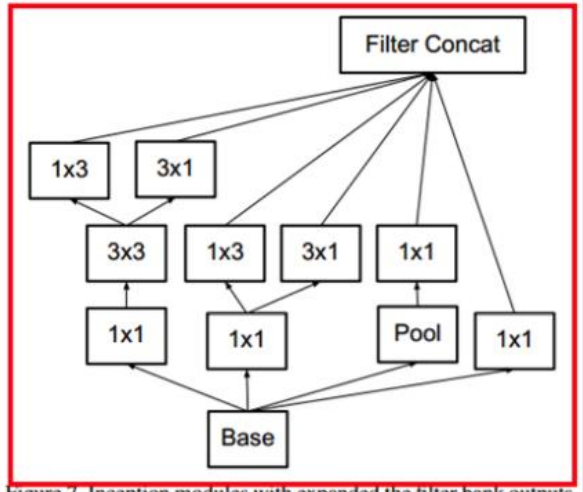


Figure 7. Inception modules with expanded the filter bank outputs. This architecture is used on the coarsest (8×8) grids to promote high dimensional representations, as suggested by principle 2 of Section 2. We are using this solution only on the coarsest grid, since that is the place where producing high dimensional sparse representation is the most critical as the ratio of local processing (by 1×1 convolutions) is increased compared to the spatial aggregation.

Inception-v4

常用的卷积神经网络

GoogLeNet (2014)

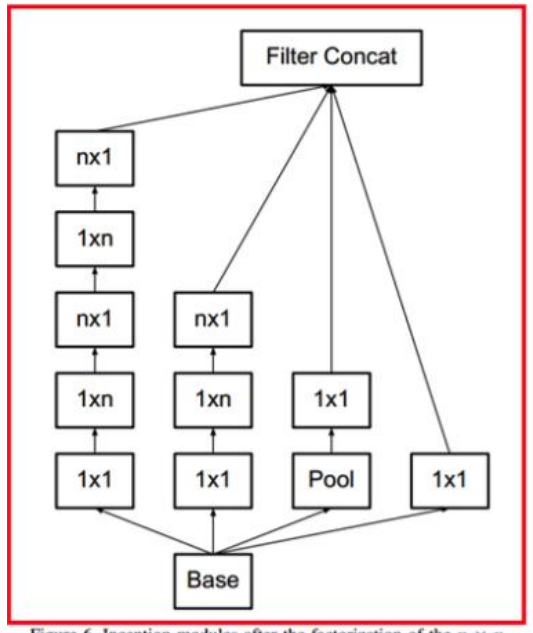


Figure 6. Inception modules after the factorization of the $n \times n$ convolutions. In our proposed architecture, we chose $n = 7$ for the 17×17 grid. (The filter sizes are picked using principle 3)

Inception-v3

```python
class InceptionC(nn.Module):
    def __init__(self, in_channels, channels_7x7):
        super(InceptionC, self).__init__()
        self.branch1x1 = BasicConv2d(in_channels, 192, kernel_size=1)

        c7 = channels_7x7
        self.branch7x7_1 = BasicConv2d(in_channels, c7, kernel_size=1)
        self.branch7x7_2 = BasicConv2d(c7, c7, kernel_size=(1, 7), padding=(0, 3))
        self.branch7x7_3 = BasicConv2d(c7, 192, kernel_size=(7, 1), padding=(3, 0))

        self.branch7x7dbl_1 = BasicConv2d(in_channels, c7, kernel_size=1)
        self.branch7x7dbl_2 = BasicConv2d(c7, c7, kernel_size=(7, 1), padding=(3, 0))
        self.branch7x7dbl_3 = BasicConv2d(c7, c7, kernel_size=(1, 7), padding=(0, 3))
        self.branch7x7dbl_4 = BasicConv2d(c7, c7, kernel_size=(7, 1), padding=(3, 0))
        self.branch7x7dbl_5 = BasicConv2d(c7, 192, kernel_size=(1, 7), padding=(0, 3))

        self.branch_pool = BasicConv2d(in_channels, 192, kernel_size=1)
    def forward(self, x):
        branch1x1 = self.branch1x1(x)

        branch7x7 = self.branch7x7_1(x)
        branch7x7 = self.branch7x7_2(branch7x7)
        branch7x7 = self.branch7x7_3(branch7x7)

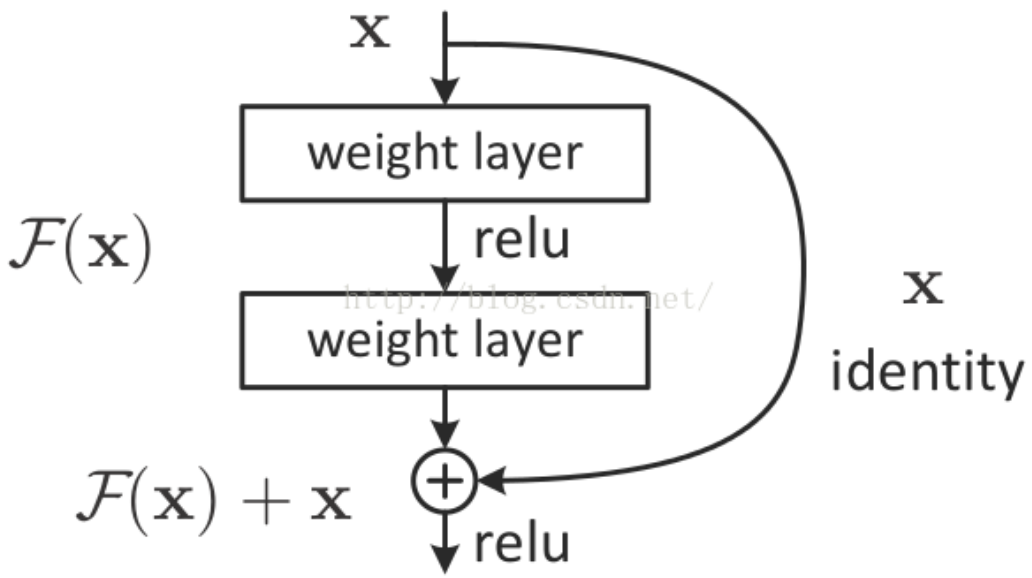
        branch7x7dbl = self.branch7x7dbl_1(x)
        branch7x7dbl = self.branch7x7dbl_2(branch7x7dbl)
        branch7x7dbl = self.branch7x7dbl_3(branch7x7dbl)
        branch7x7dbl = self.branch7x7dbl_4(branch7x7dbl)
        branch7x7dbl = self.branch7x7dbl_5(branch7x7dbl)

        branch_pool = F.avg_pool2d(x, kernel_size=3, stride=1, padding=1)
        branch_pool = self.branch_pool(branch_pool)

        outputs = [branch1x1, branch7x7, branch7x7dbl, branch_pool]
        return torch.cat(outputs, 1)
```

常用的卷积神经网络

ResNet (2016)

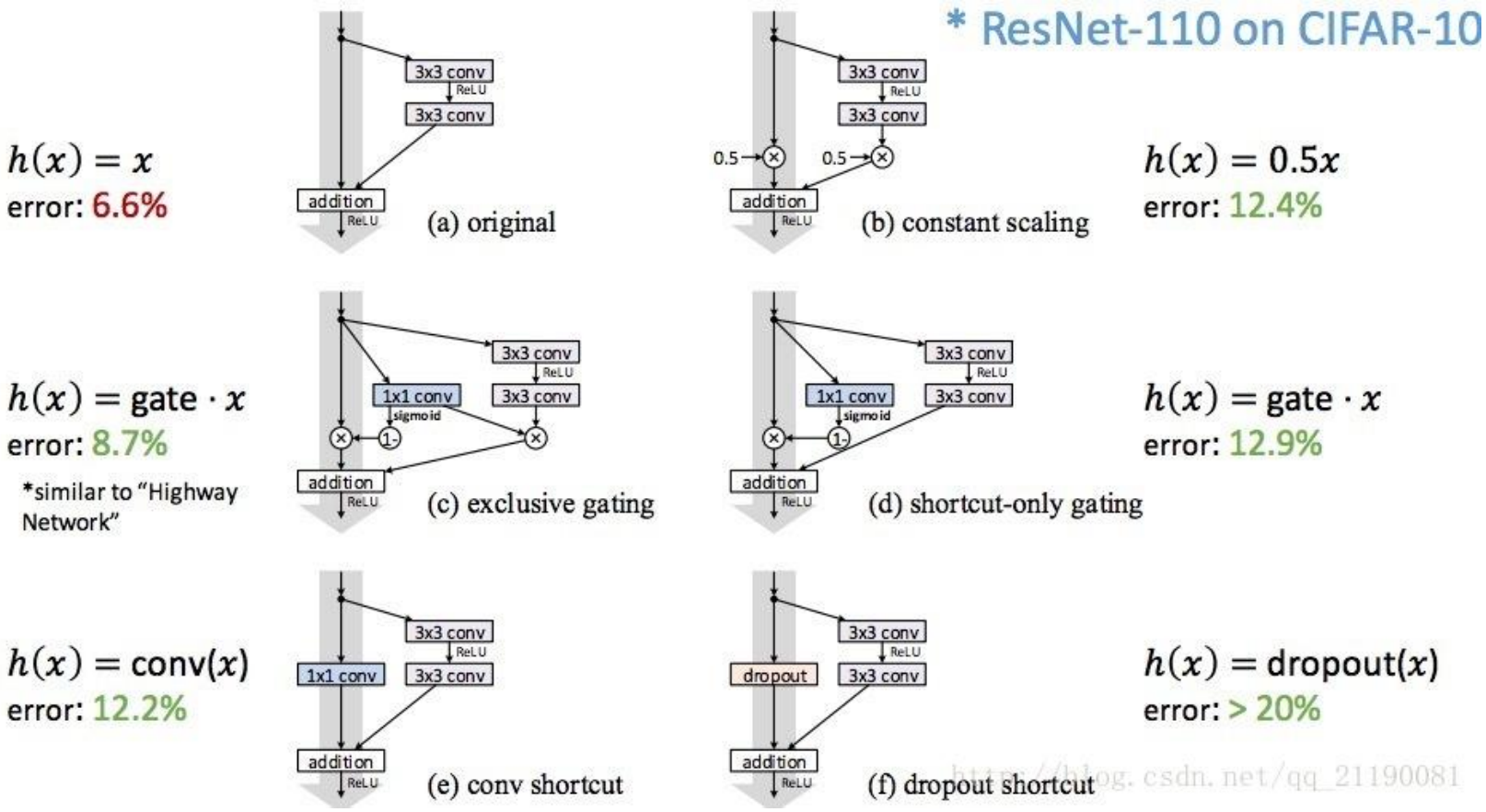


Residual block

- Resnet的出发点是认为深层网络不应该比浅层网络性能差，所以为了防止网络退化，引入了大量identity恒等映射，这样就可以把原始信息流入更深的层，抑制了信息的退化
- 残差块有用是因为identity这一支路的导数是1，所以可以把深层的loss很好的保留传递给浅层，因为神经网络一个很大的问题就是梯度链式法则带来的梯度弥散
- 残差块就是一个差分放大器
- ...

常用的卷积神经网络

ResNet (2016)



https://blog.csdn.net/qq_21190081

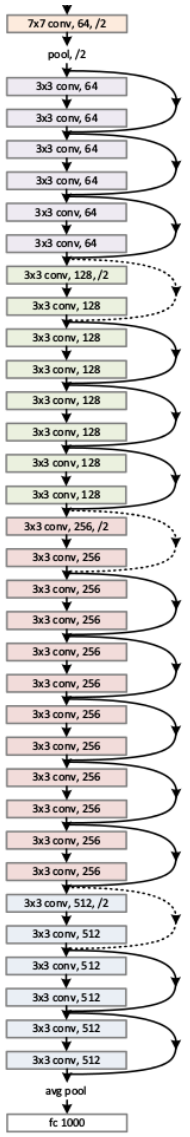
Shortcut支路就只能是identity恒等映射，用其他映射效果都不如identity

常用的卷积神经网络

ResNet (2016)

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

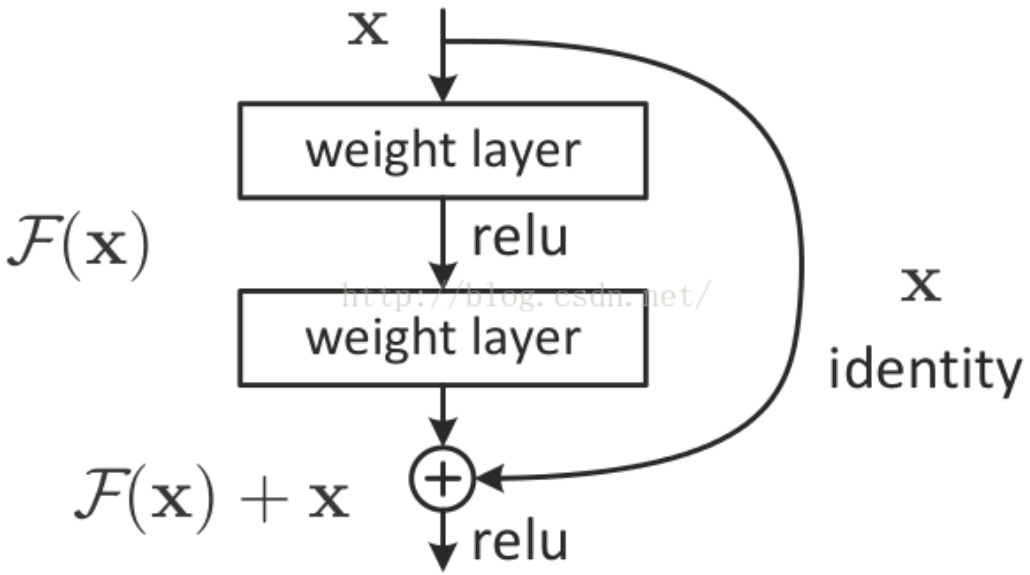
http://blog.csdn.net/u014421190081



ResNet最多可到1001层

常用的卷积神经网络

Resnet (2016)



Residual block

```python
def conv3x3(in_planes, out_planes, stride=1):
    """3x3 convolution with padding"""
    return nn.Conv2d(in_planes, out_planes, kernel_size=3, stride=stride,
                     padding=1, bias=False)

class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, inplanes, planes, stride=1, downsample=None):
        super(BasicBlock, self).__init__()
        self.conv1 = conv3x3(inplanes, planes, stride)
        self.bn1 = nn.BatchNorm2d(planes)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = conv3x3(planes, planes)
        self.bn2 = nn.BatchNorm2d(planes)
        self.downsample = downsample
        self.stride = stride

    def forward(self, x):
        residual = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)

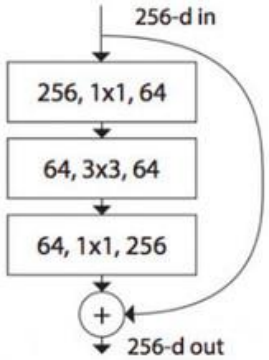
        if self.downsample is not None:
            residual = self.downsample(x)

        out += residual
        out = self.relu(out)

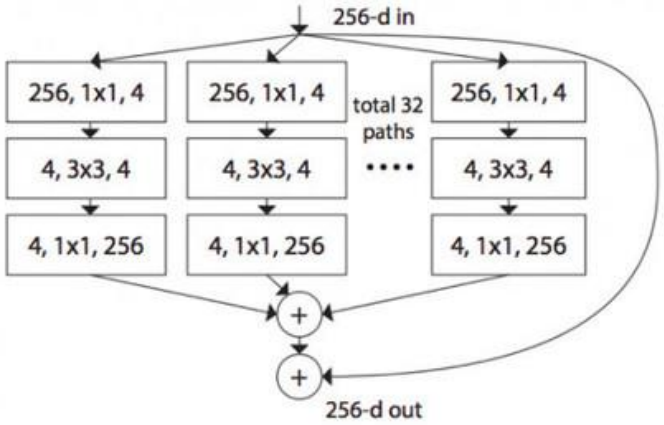
        return out
```

常用的卷积神经网络

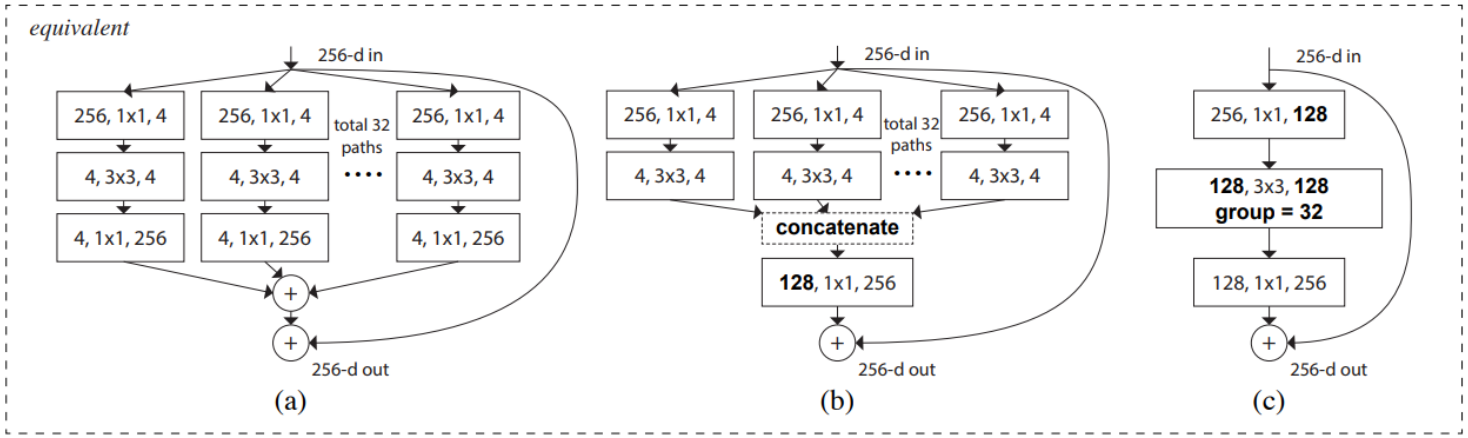
ResNeXt (2017)



ResNet



ResNeXt

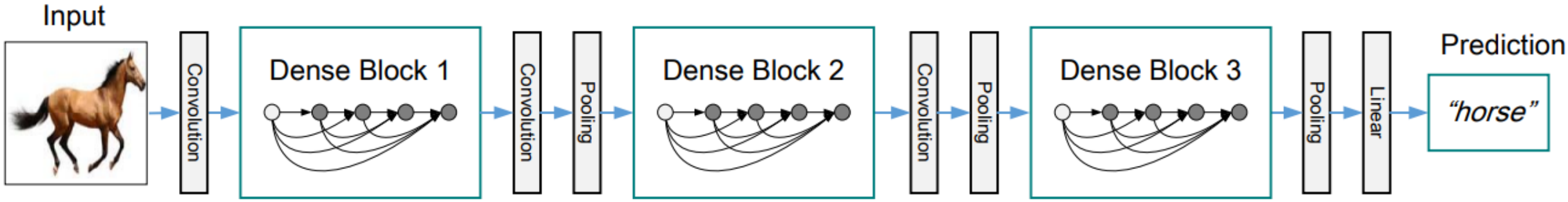


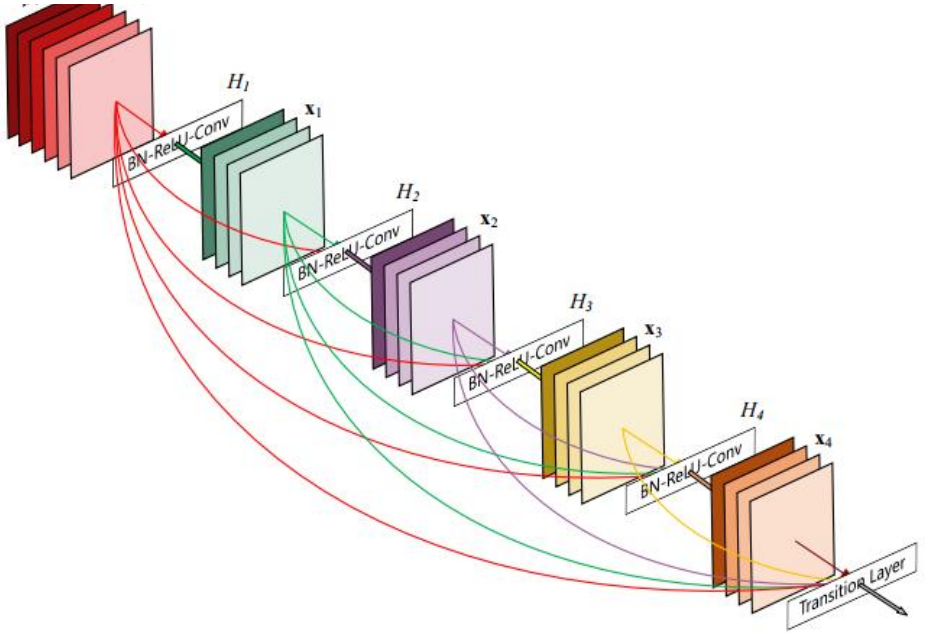
Equivalent

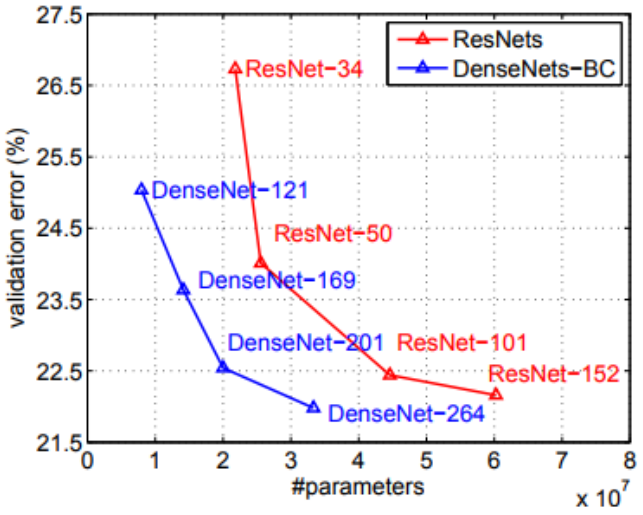
ResNeXt在ResNet里面引入分组卷积的思想

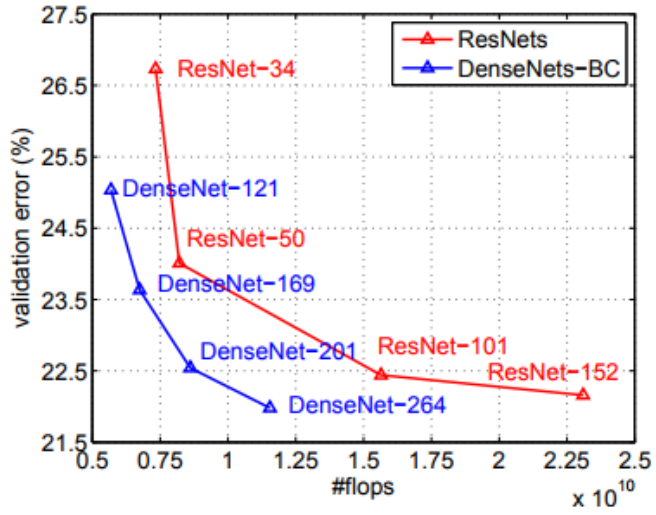
常用的卷积神经网络

DenseNet (2017)





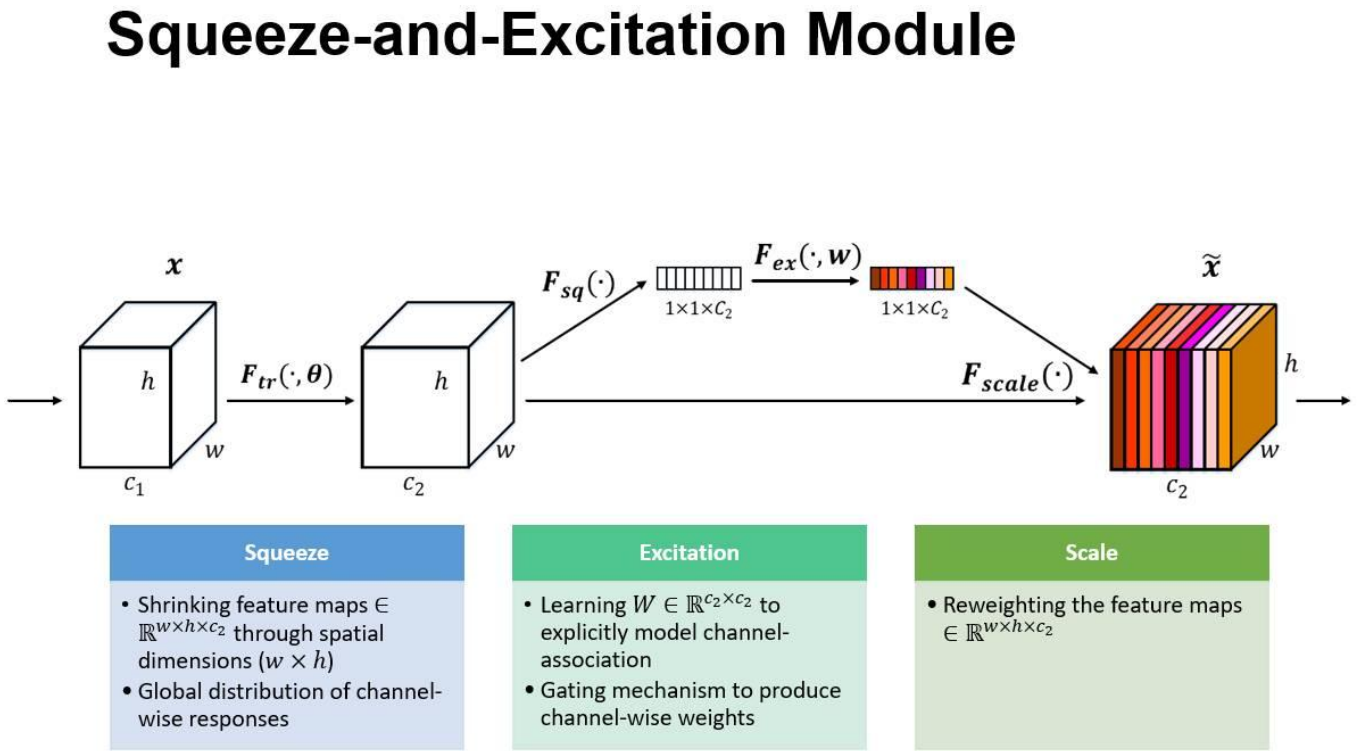


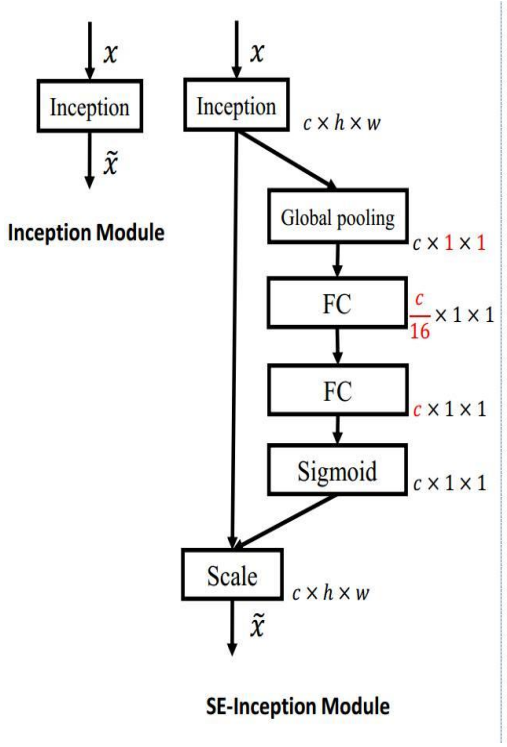


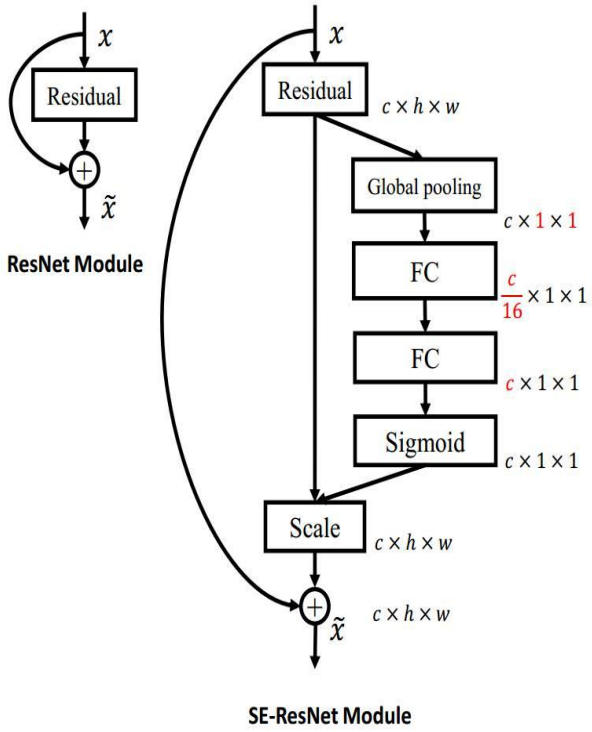
DenseNet把shortcut思想发挥到了极致

常用的卷积神经网络

SENet(2017)



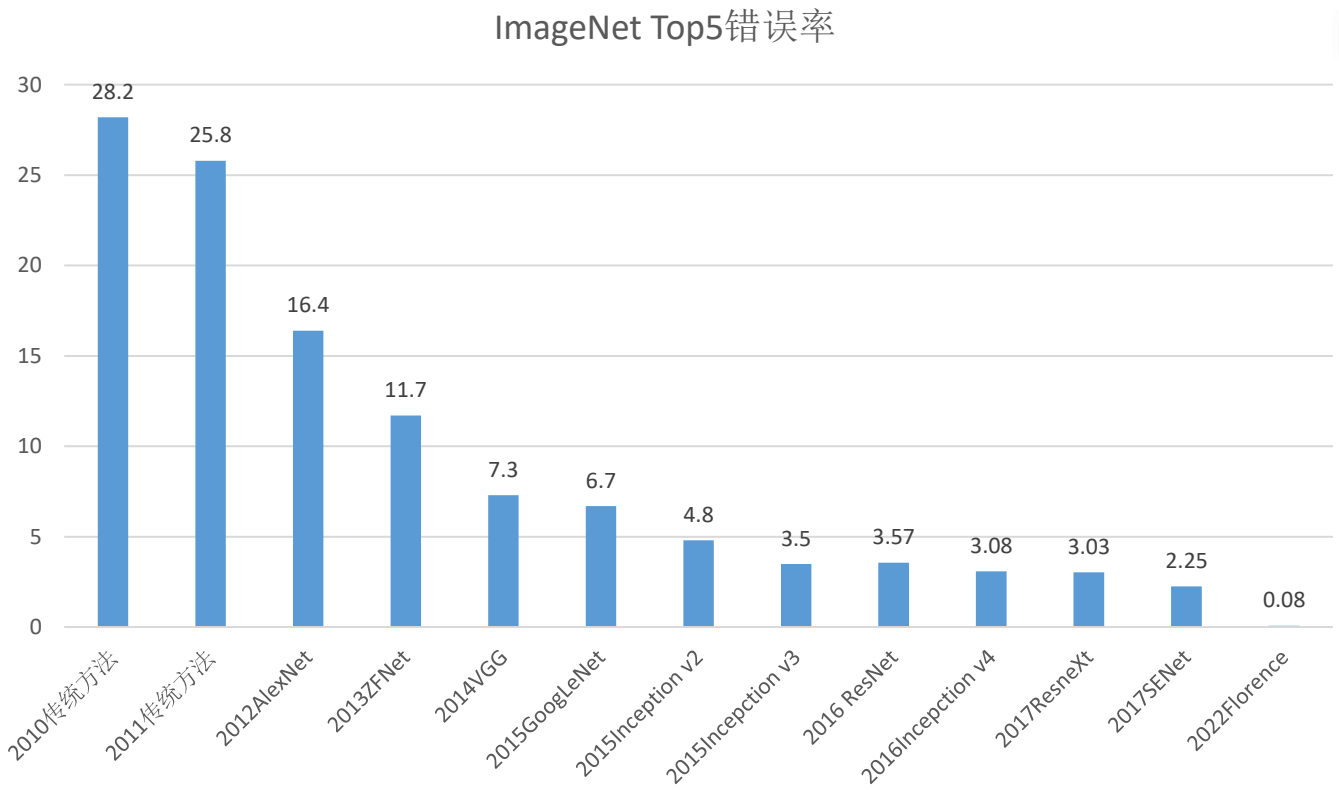


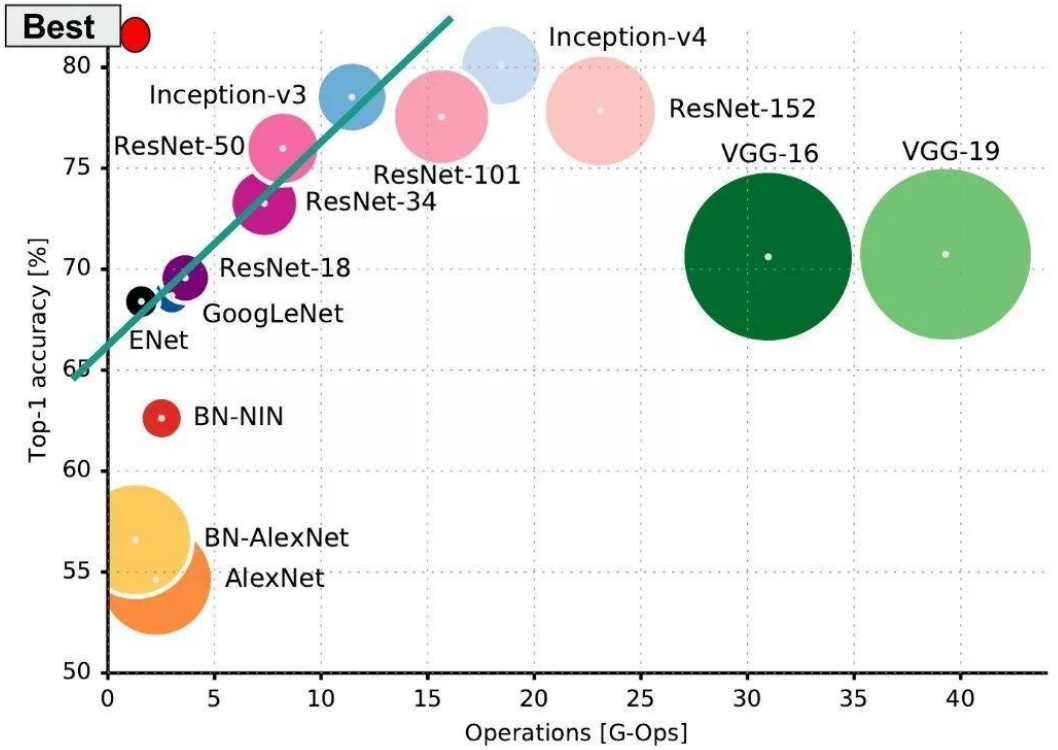


- 融合spatial信息和channel-wise信息
- SE module的核心就是一个channel attention

常用的卷积神经网络

Summary





常用的卷积神经网络

常用网络计算量

Year	Model	Layers	Parameter	FLOPs	ImageNet Top-5 error
2012	AlexNet	5+3	60M	725M	16.4%
2013	Clarifai	5+3	60M	—	11.7%
2014	MSRA	5+3	200M	—	8.06%
2014	VGG-19	16+3	143M	19.6B	7.32%
2014	GoogLeNet	22	6.8M	1.566B	6.67%
2015	ResNet	152	19.4M	11.3B	3.57%

第 3 部分

模型压缩与加速概述

模型压缩与加速概述

深度学习的困境

移动设备：算不好







穿戴设备：算不了







数据中心：算不起



模型压缩与加速概述

理论基础

事实上当今许多深度学习的应用都是在资源受限的设备上！！

- **必要性**

在许多网络结构中，如VGG-16网络，参数数量1亿3千多万，占用500MB空间，需要进行309亿次浮点运算才能完成一次图像识别任务。
伴随着神经网络的发展，高级卷积网络架构和Transformers给视觉和语言领域任务带来的巨大提升的同时，模型的规模也越来越大。如DeepSeek-R1模型，参数数量高达6710亿，占用约404GB的存储空间，需要多张A100/V100显卡同时运行才能完成一次复杂的推理任务。
虽然模型体积的增加往往能带来更好的预测性能，但是这也必然增加了模型开发和应用成本，甚至导致在现实业务场景中的可用性几乎为0。

- **可行性**

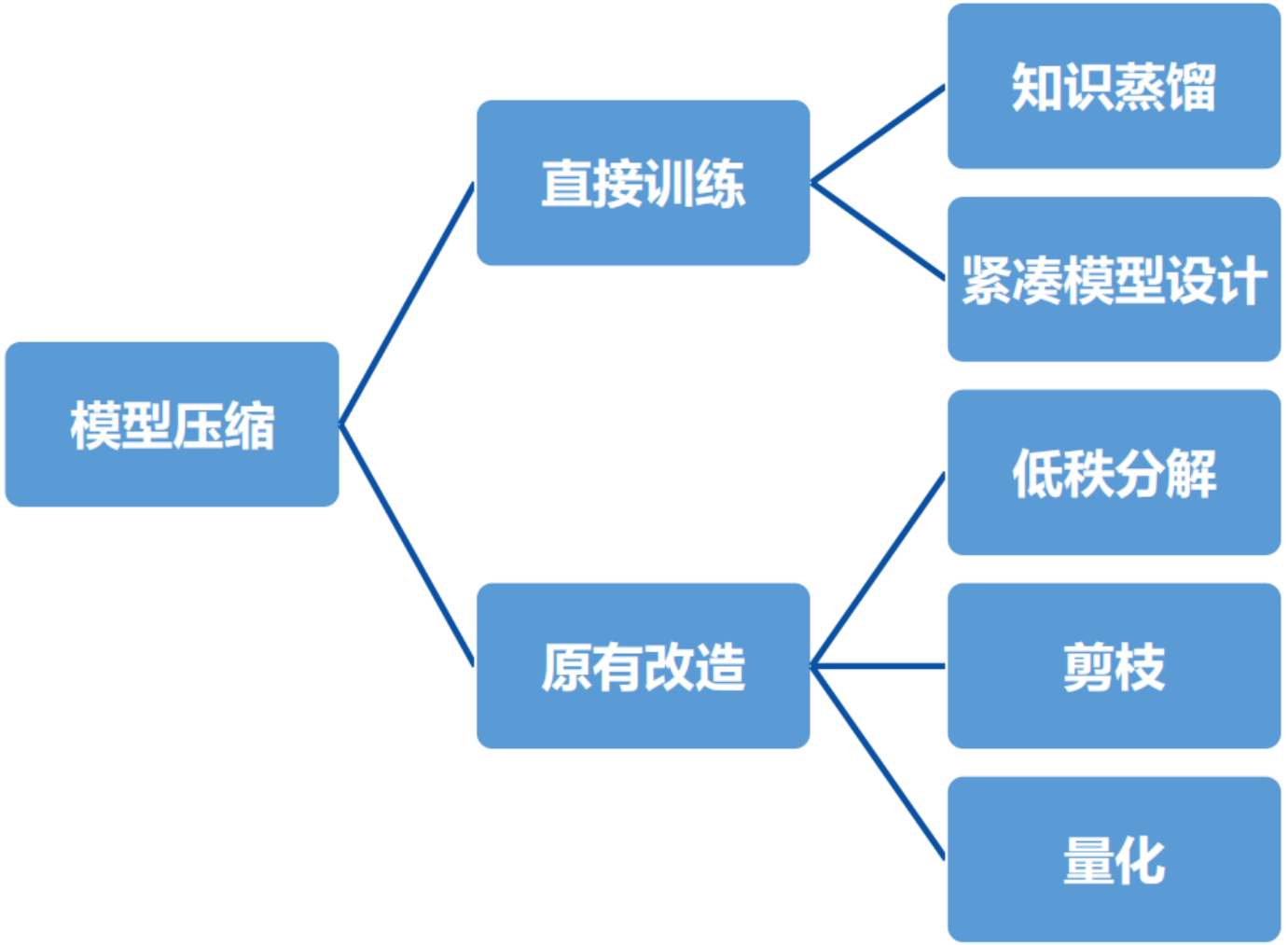
论文<Predicting parameters in deep learning>提出，其实在很多深度的神经网络中存在着显著的冗余。仅仅使用很少一部分（5%）权值就足以预测剩余的权值。该论文还提出这些剩下的权值甚至可以直接不用被学习。也就是说，仅仅训练一小部分原来的权值参数就有可能达到和原来网络相近甚至超过原来网络的性能（可以看作一种正则化）。

- **最终目的**

最大程度的减小模型复杂度，减少模型存储需要的空间，也致力于加速模型的训练和推测

模型压缩与加速概述

方法概述

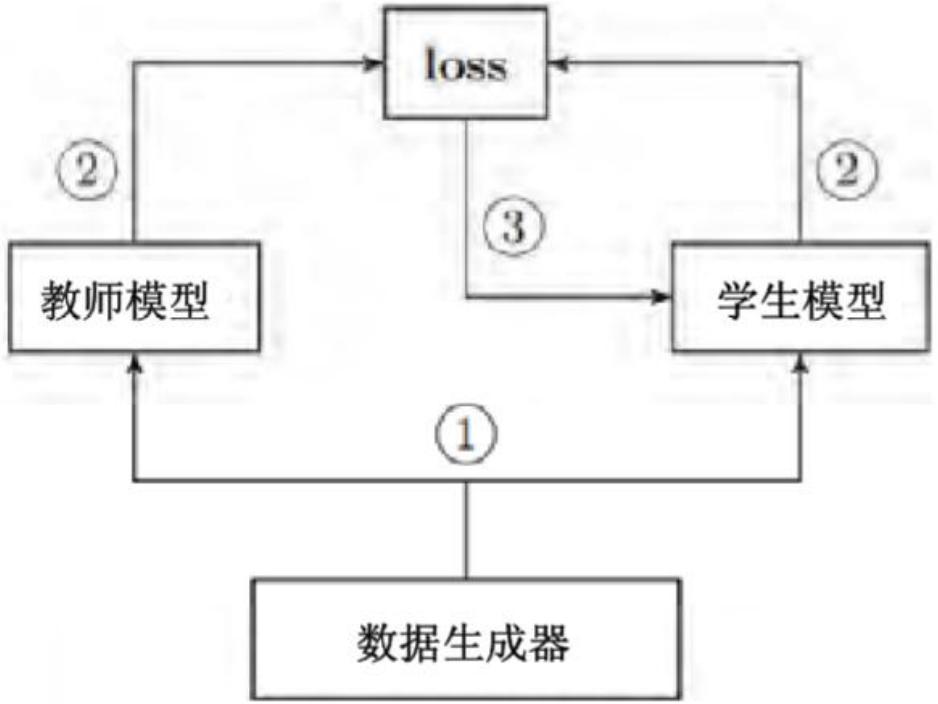


第 4 部分

模型压缩与加速方法

模型压缩与加速方法

知识蒸馏（knowledge distillation）



蒸馏模型采用的是迁移学习，通过采用预先训练好的教师模型（Teacher model）的输出作为监督信号去训练另外一个轻量化的网络（ Student model ）。

教师模型：大&强

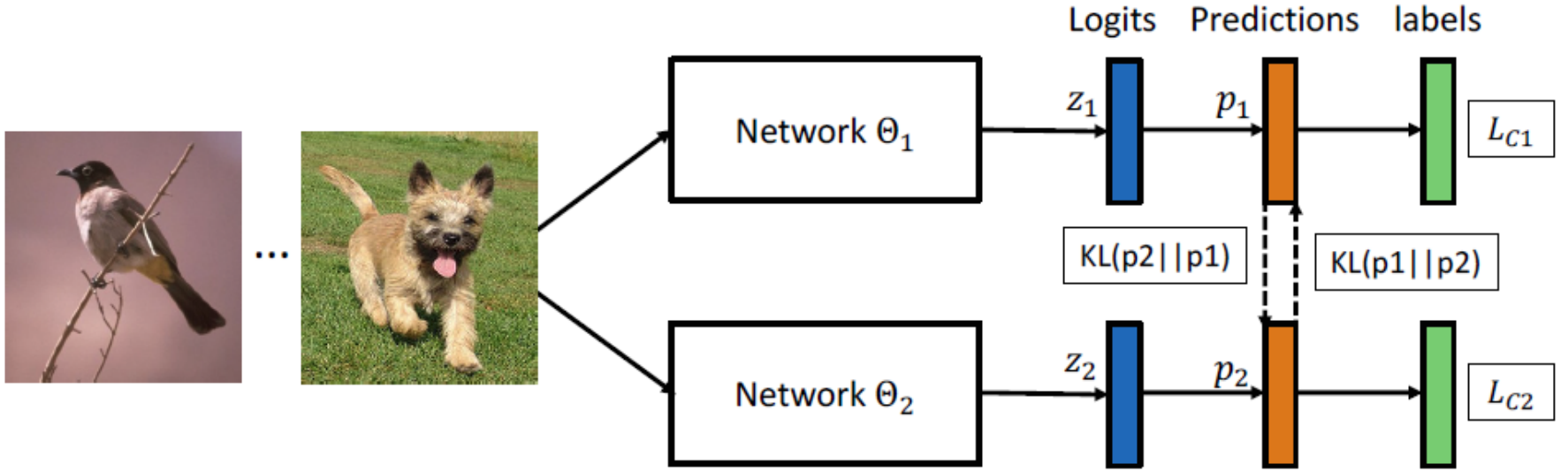
学生模型：小&快

监督的信息可以为：

- 概率分布
- 输出的特征
- 中间层feature map
- Attention map
- 中间过程

知识蒸馏（KD）

概率分布



$$q_i = \frac{exp(z_i/T)}{\sum_j exp(z_j/T)}$$

$$D_{KL}(\boldsymbol{p}_2\|\boldsymbol{p}_1) = \sum_{i=1}^{N}\sum_{m=1}^{M} p_2^m(\boldsymbol{x}_i)\log\frac{p_2^m(\boldsymbol{x}_i)}{p_1^m(\boldsymbol{x}_i)}.$$

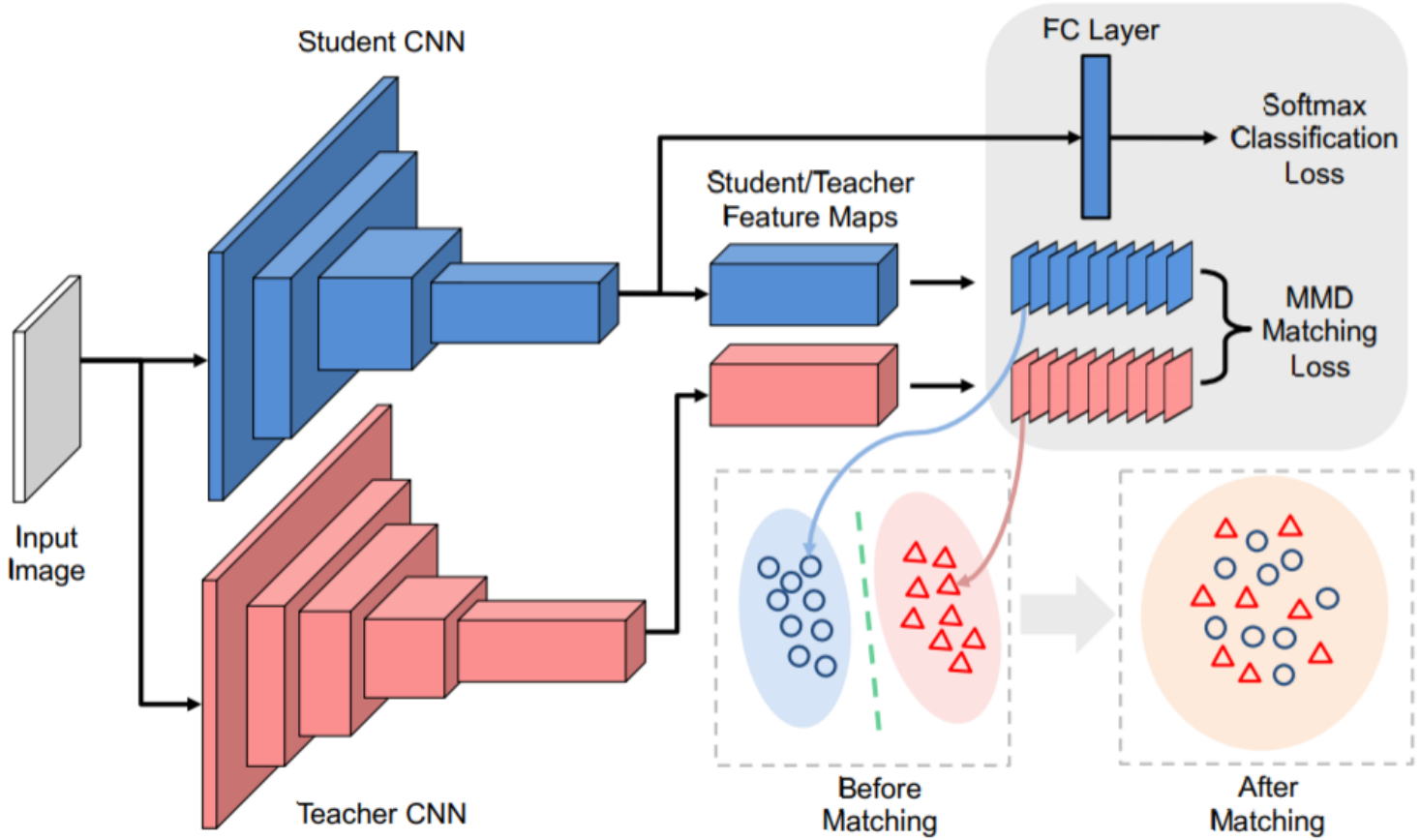
$$L_{\Theta_1} = L_{C_1} + D_{KL}(\boldsymbol{p}_2\|\boldsymbol{p}_1)$$

KL散度：衡量两个概率分布之间有多大差异了

torch.nn.KLDivLoss(*size_average=None*, *reduce=None*, *reduction='elementwise_mean'*)

知识蒸馏（KD）

Feature Map——MMD Loss



MMD Loss：最大均值差异（Maximum Mean Discrepancy）损失，可以评价两个域的相似性

$$MMD(X,Y) = ||\frac{1}{n}\sum_{i=1}^{n}\phi(x_i) - \frac{1}{m}\sum_{j=1}^{m}\phi(y_j)||_H^2$$

$\phi()$ 将数据映射到再生希尔伯特空间（RKHS）中进行度量

知识蒸馏（KD）

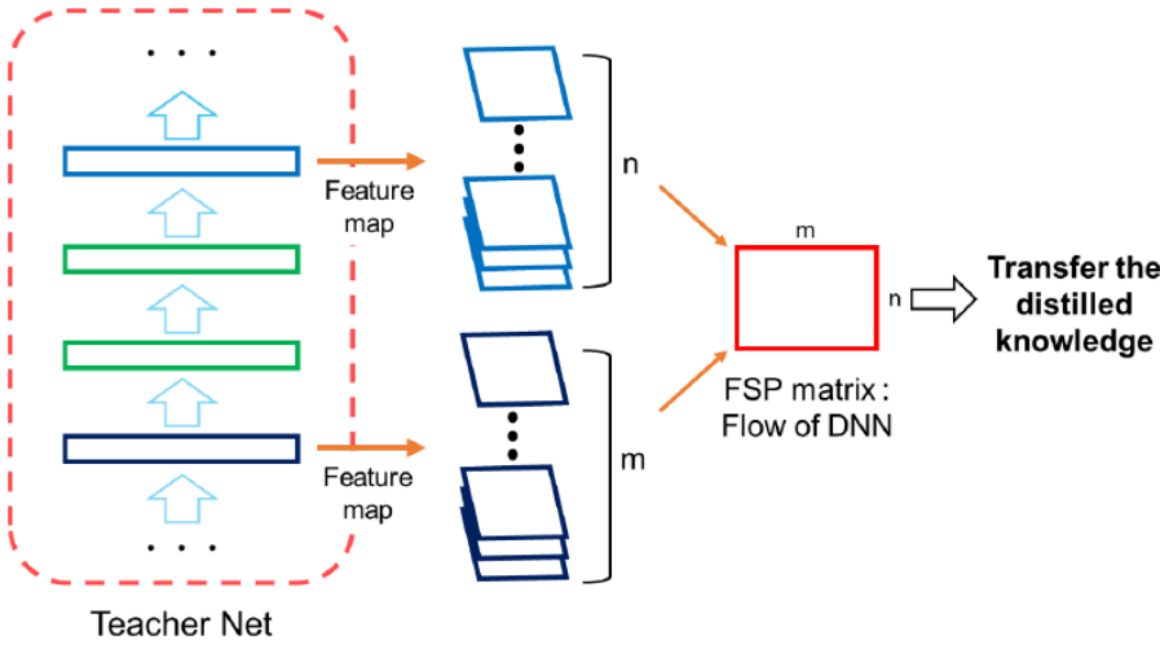
中间过程——L2损失

- Knowledge Definition: (FSP) flow of solution procedure matrix

$$G_{i,j}(x;W) = \sum_{s=1}^{h}\sum_{t=1}^{w} \frac{F^1_{s,t,i}(x;W) \times F^2_{s,t,j}(x;W)}{h \times w}$$

- Loss Definition: L2 loss

$$L_{FSP}(W_t, W_s) = \frac{1}{N}\sum_{x}\sum_{i=1}^{n} \lambda_i \times \|(G^T_i(x;W_t) - G^S_i(x;W_s)\|^2_2,$$

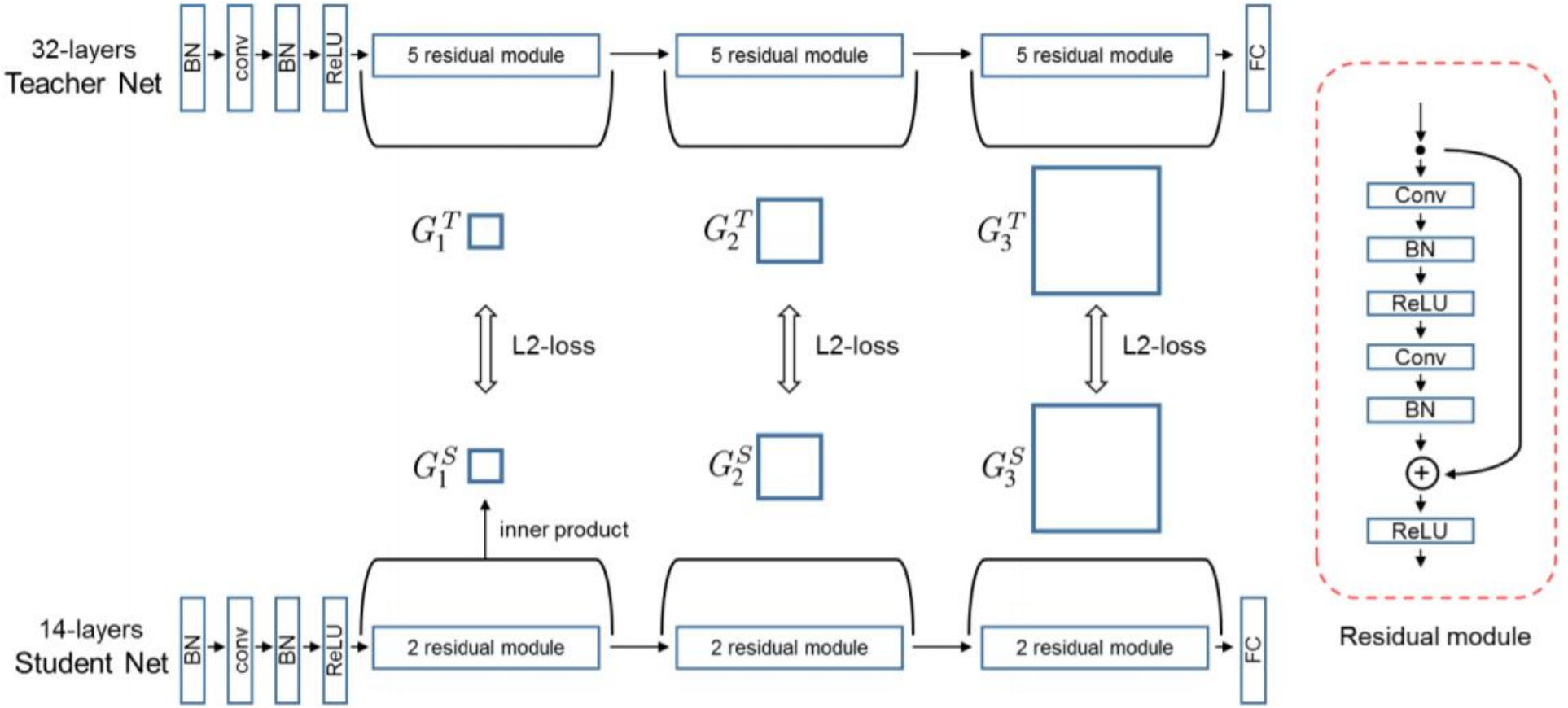


利用Gram矩阵来拟合层与层之间的关系

torch.nn.MSELoss(*size_average=None, reduce=None, reduction='elementwise_mean'*)

知识蒸馏（KD）

中间过程——L2损失



Yim J, Joo D, Bae J, et al. A gift from knowledge distillation: Fast optimization, network minimization and transfer learning[C]//The IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2017, 2.

模型压缩与加速方法

紧致的网络结构设计

常见的紧致网络：

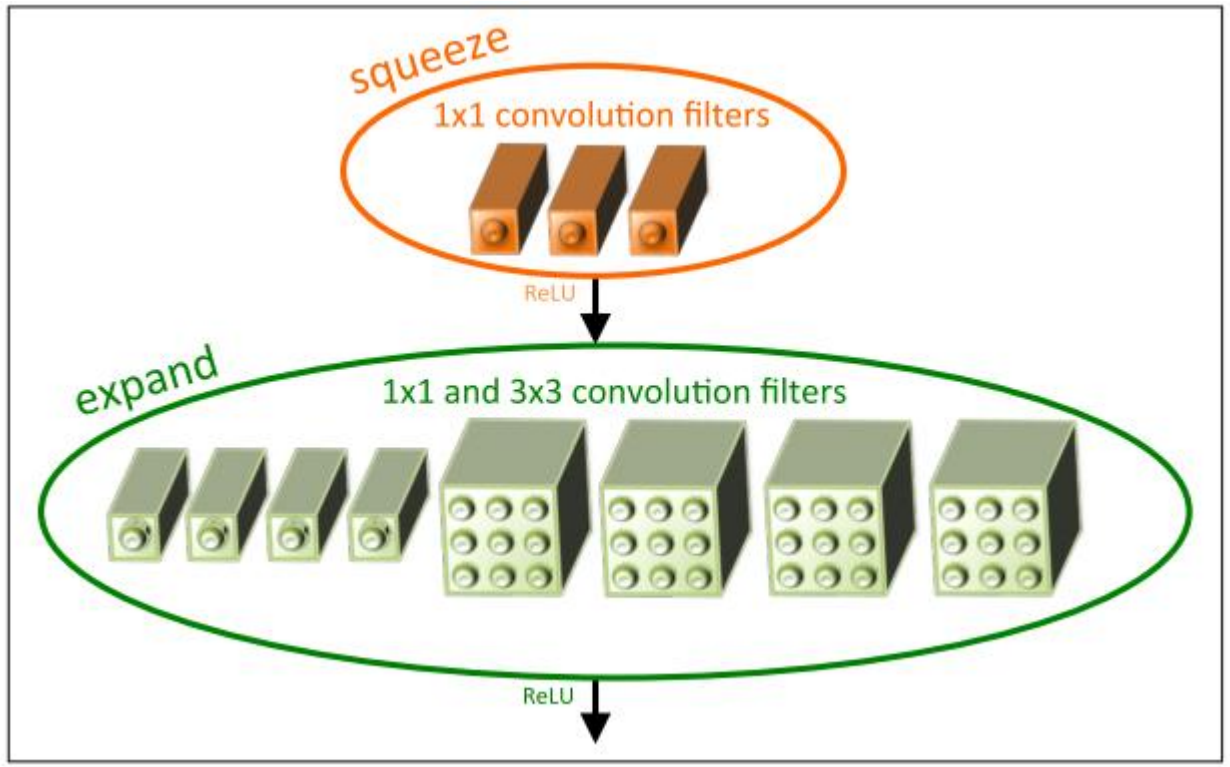
- ShuffleNet
- MobileNet v1
- MobileNet v2
- SqueezeNet

Model	
SqueezeNet	1*1 filters , input channels , Downsample late
MobileNet v1	DWconv + Pconv ,Width Multiplier ,Resolution Multiple
MobileNet v2	Inverted residuals ,Linear bottlenecks
ShuffleNet	DWconvolution with Channel Shuffle

直接设计又小又快又好的网络，这就是紧致网络设计（Compact Network Design）的方法

紧致的网络结构设计

SqueezeNet




```python
class Fire(nn.Module):

    def __init__(self, inplanes, squeeze_planes,
                 expand1x1_planes, expand3x3_planes):
        super(Fire, self).__init__()
        self.inplanes = inplanes
        self.squeeze = nn.Conv2d(inplanes, squeeze_planes, kernel_size=1)
        self.squeeze_activation = nn.ReLU(inplace=True)
        self.expand1x1 = nn.Conv2d(squeeze_planes, expand1x1_planes,
                                   kernel_size=1)
        self.expand1x1_activation = nn.ReLU(inplace=True)
        self.expand3x3 = nn.Conv2d(squeeze_planes, expand3x3_planes,
                                   kernel_size=3, padding=1)
        self.expand3x3_activation = nn.ReLU(inplace=True)

    def forward(self, x):
        x = self.squeeze_activation(self.squeeze(x))
        return torch.cat([
            self.expand1x1_activation(self.expand1x1(x)),
            self.expand3x3_activation(self.expand3x3(x))
        ], 1)
```

Fire Module

紧致的网络结构设计

SqueezeNet

1.使用更小的1*1卷积核来替换3*3卷积核

2.减少输入3*3卷积的特征图的数量

3.减少pooling

4.比起AlexNet，SqueezeNet可以压缩510x的情况下在ImageNet上得到类似精度

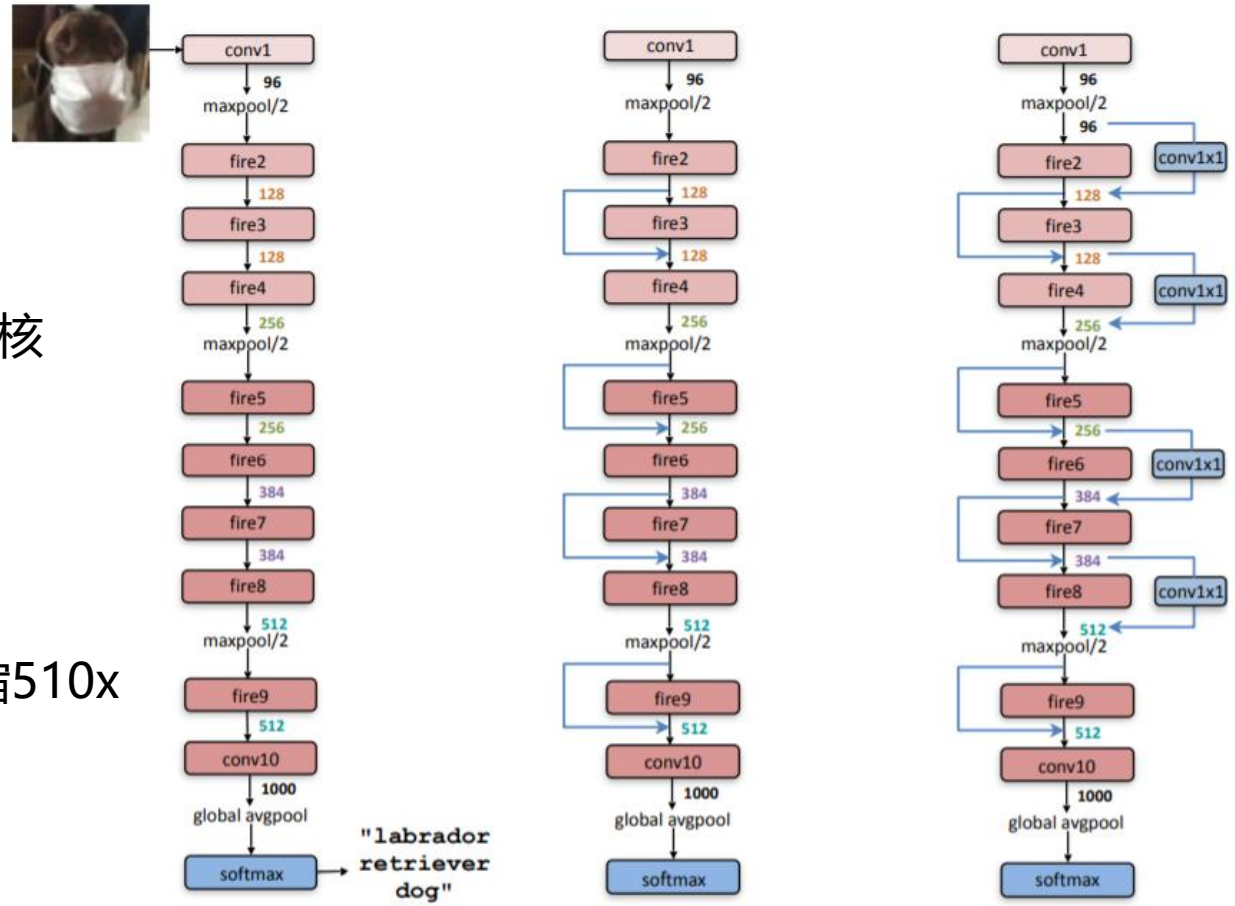
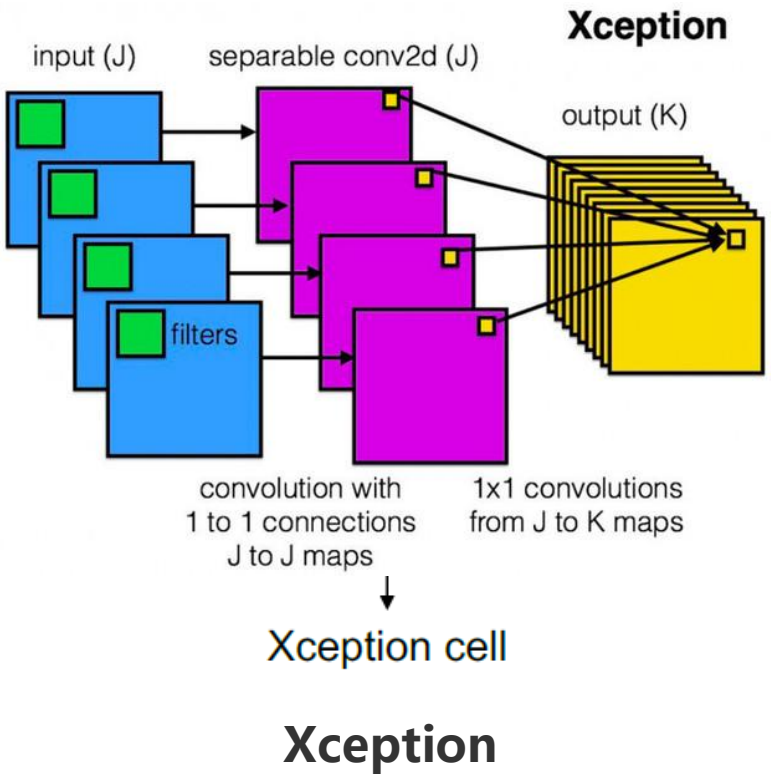


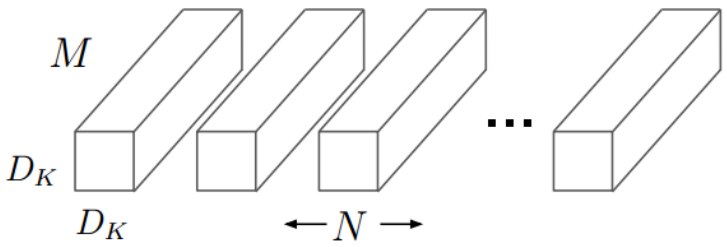
Figure 2: Macroarchitectural view of our SqueezeNet architecture. Left: SqueezeNet (Section 3.3); Middle: SqueezeNet with simple bypass (Section 6); Right: SqueezeNet with complex bypass (Section 6).

紧致的网络结构设计

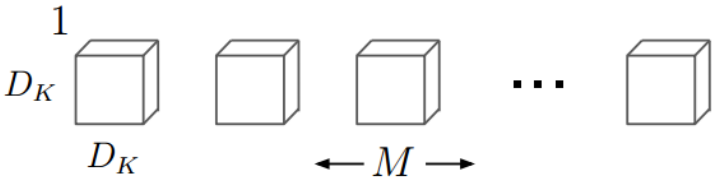
Xception结构



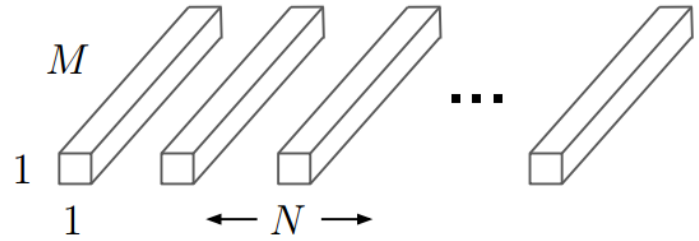
Xception



(a) Standard Convolution Filters



(b) Depthwise Convolutional Filters



(c) 1×1 Convolutional Filters called Pointwise Convolution in the context of Depthwise Separable Convolution

http://blog.csdn.net/

把普通卷积操作分成两部分

- Depthwise Convolution

  计算量 $D_K \cdot D_K \cdot M \cdot D_F \cdot D_F$

- Pointwise Convolution

  计算量 $M \cdot N \cdot D_F \cdot D_F$

上面两步合称Depthwise Separable Convolution

与原卷积计算量之比 $\frac{D_K \cdot D_K \cdot M \cdot D_F \cdot D_F + M \cdot N \cdot D_F \cdot D_F}{D_K \cdot D_K \cdot M \cdot N \cdot D_F \cdot D_F} = \frac{1}{N} + \frac{1}{D_K^2}$

- MobileNet里面利用了大量的Xception结构
- Xception使计算量减少，但是显存消耗增多

紧致的网络结构设计

MobileNet

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
$5\times$ Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Table 2. Resource Per Layer Type

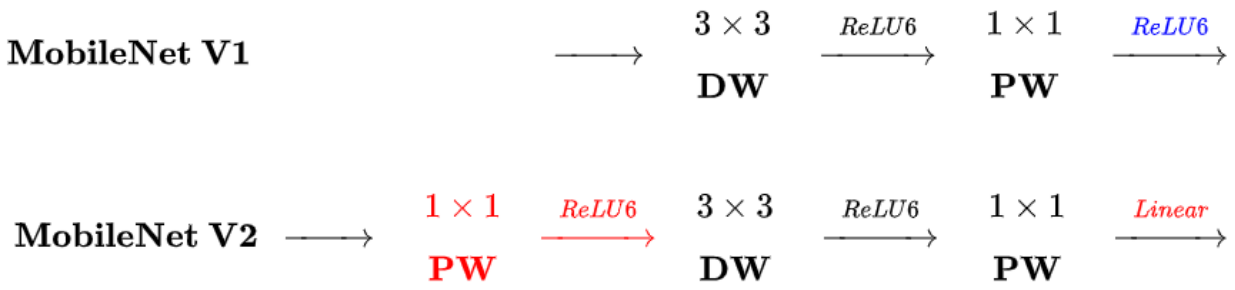
Type	Mult-Adds	Parameters
Conv 1×1	94.86%	74.59%
Conv DW 3×3	3.06%	1.06%
Conv 3×3	1.19%	0.02%
Fully Connected	0.18%	24.33%

- MobileNet 是 Xception 结构的级联
- 大量网络消耗集中在1×1的Pointwise卷积上

Howard A G, Zhu M, Chen B, et al. Mobilenets: Efficient convolutional neural networks for mobile vision applications[J]. arXiv preprint arXiv:1704.04861, 2017.

紧致的网络结构设计

MobileNet v2



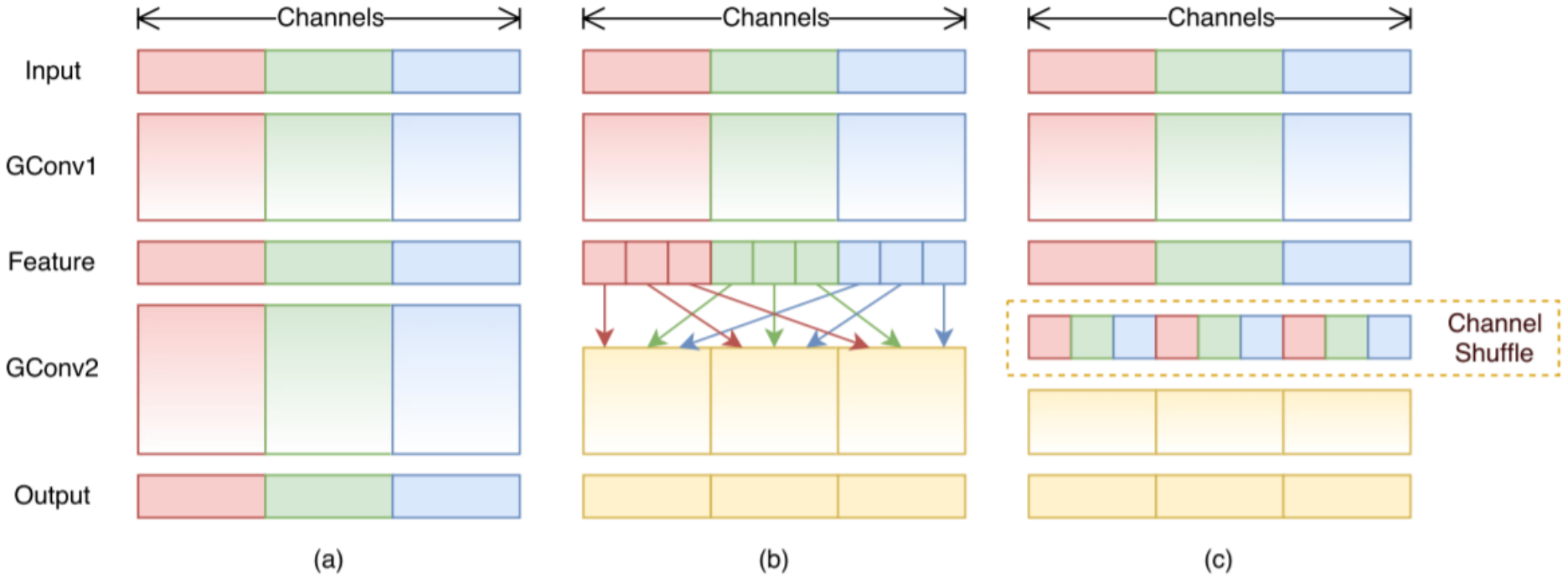
Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$28^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times k$	conv2d 1x1	-	k	-	

- V2 在 DW 卷积之前新加了一个 1×1卷积，增加网路通道数。
- V2 去掉了第二个 PW 的激活函数，防止特征丢失。
- V2增加了ResNet的shortcut结构

- 通过改变网络的参数设置，V2比V1进一步压缩了约50%，并且泛化能力提升。

紧致的网络结构设计

ShuffleNet



- ShuffleNet使用通道打乱操作来代替1×1卷积，实现通道信息的融合
- Shuffle操作通过Reshape操作实现，不包含参数

紧致的网络结构设计

ShuffleNet

```python
class ChannelShuffle(nn.Module):
    def __init__(self, num_groups):
        super(ChannelShuffle, self).__init__()
        self.g = num_groups

    def forward(self, x):
        b, c, h, w = x.size()
        n = c / self.g
        # reshape
        x = x.view(b, self.g, n, h, w)
        # transpose
        x = x.permute(0, 2, 1, 3, 4).contiguous()
        # flatten
        x = x.view(b, c, h, w)
        return x
```

```
>>> x = torch.randn(2, 3, 5)
>>> x.size()
torch.Size([2, 3, 5])
>>> x.permute(2, 0, 1).size()
torch.Size([5, 2, 3])
```

```
In [1]: a = [[1,2,3],[4,5,6],[7,8,9]]

In [2]: import torch

In [3]: x = torch.tensor(a)

In [4]: x
Out[4]:
tensor([[ 1,  2,  3],
        [ 4,  5,  6],
        [ 7,  8,  9]])

In [5]: y = x.permute(1,0)

In [6]: y
Out[6]:
tensor([[ 1,  4,  7],
        [ 2,  5,  8],
        [ 3,  6,  9]])
```

Permute操作可以交换维度

紧致的网络结构设计

ShuffleNet

Model	Cls err. (%)	Complexity (MFLOPs)
VGG-16 [30]	28.5	15300
ShuffleNet 2× ($g = 3$)	26.3	**524**
GoogleNet [33]*	31.3	1500
ShuffleNet 1× ($g = 8$)	32.4	**140**
AlexNet [21]	42.8	720
SqueezeNet [14]	42.5	833
ShuffleNet 0.5× ($g = 4$)	41.6	**38**

Model	Cls err. (%)	FLOPs	224 × 224	480 × 640	720 × 1280
ShuffleNet 0.5× ($g = 3$)	43.2	38M	15.2ms	87.4ms	260.1ms
ShuffleNet 1× ($g = 3$)	32.6	140M	37.8ms	222.2ms	684.5ms
ShuffleNet 2× ($g = 3$)	26.3	524M	108.8ms	617.0ms	1857.6ms
AlexNet [21]	42.8	720M	184.0ms	1156.7ms	3633.9ms
1.0 MobileNet-224 [12]	29.4	569M	110.0ms	612.0ms	1879.2ms

Xiangyu Z, Xinyu Z, Mengxiao L, et al. Shufflenet: an extremely efficient convolutional neural network for mobile devices[C]//Computer Vision and Pattern Recognition. 2017.

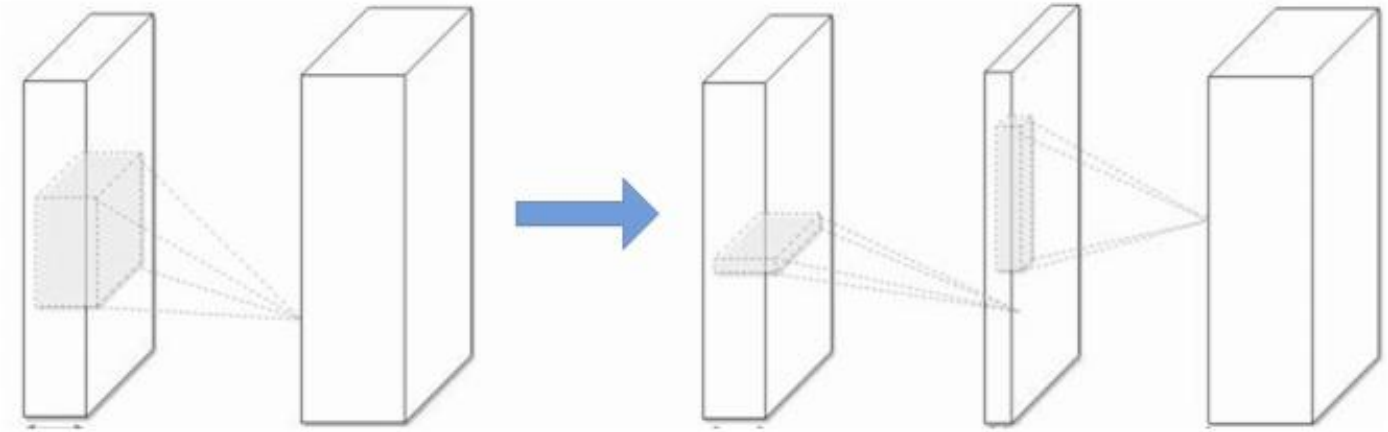
紧致的网络结构设计

紧致的网络结构使用情况对比

Model	Caffe	Tensorflow	Keras	PyTorch	Migration network	Recommendation level
SqueezeNet	1★	4★	2★	2★	AlexNet,faster-rcnn	3★
MobileNet v1	4★	5★	3★	3★	Mobilenet-SSD, MXNet,faster-rcnn	3★
MobileNet v2	3★	5★	2★	4★	MobileNetv2-SSDLite	5★
ShuffleNet	5★	4★	2★	3★	Shufflenet-SSD,	4★

模型压缩与加速方法

低秩近似（Low Rank）



一个典型的 CNN 卷积核是一个 4D 张量，需要注意的是这些张量中可能存在大量的冗余。而基于张量分解的思想也许是减少冗余的很有潜力的方法。而全连接层也可以当成一个 2D 矩阵，低秩分解同样可行。

低秩近似(Low Rank)

卷积分解

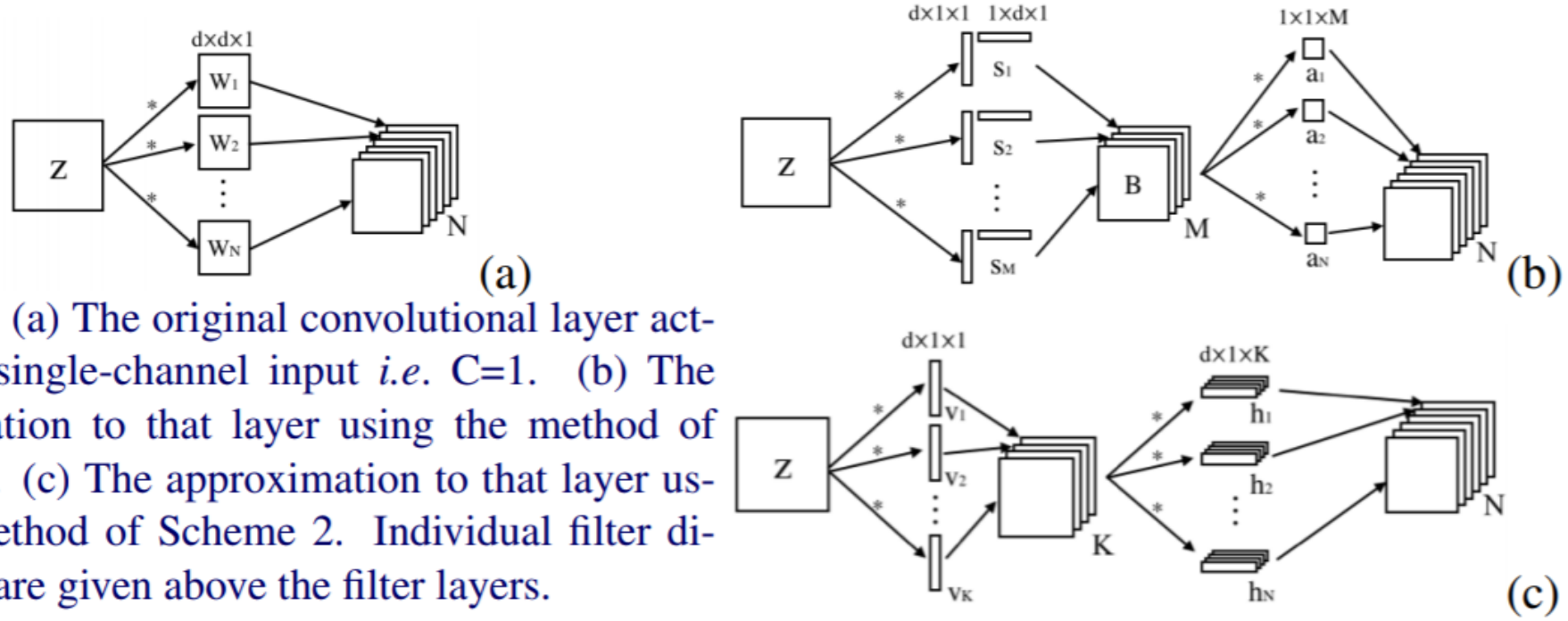


Figure 1: (a) The original convolutional layer acting on a single-channel input *i.e.* C=1. (b) The approximation to that layer using the method of Scheme 1. (c) The approximation to that layer using the method of Scheme 2. Individual filter dimensions are given above the filter layers.

- 将d×d卷积核分解为d×1和1×d卷积核
- 损失精度小于1%

低秩近似(Low Rank)

SVD分解

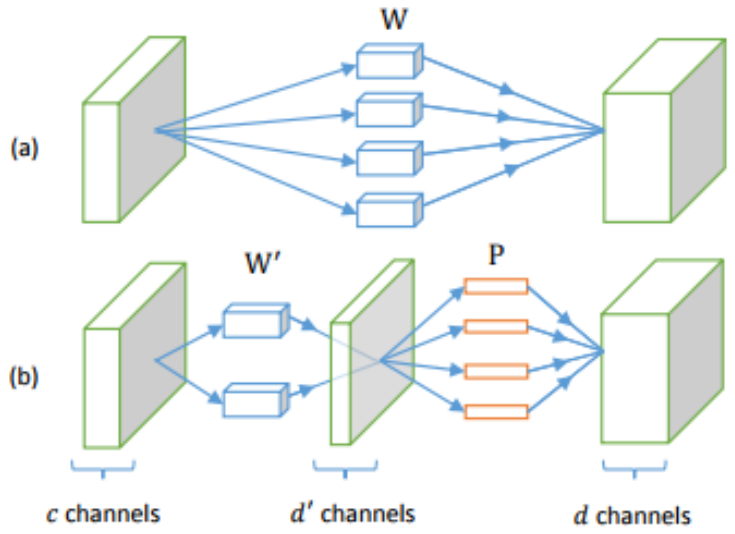


Figure 1: Illustration of the decomposition. (a) An original layer with complexity $O(dk^2c)$. (b) An approximated layer with complexity reduced to $O(d'k^2c) + O(dd')$.

SVD分解

- 对矩阵进行SVD分解，保留特征值最大的几项

$$\mathbf{y} = \mathrm{M}(\mathbf{y} - \bar{\mathbf{y}}) + \bar{\mathbf{y}}.$$

$$\mathbf{y} = \mathrm{MW}\mathbf{x} + \mathbf{b},$$

$$\mathbf{b} = \bar{\mathbf{y}} - \mathrm{M}\bar{\mathbf{y}}$$

$$\mathrm{M} = \mathrm{PQ}^{\top}$$

$$\mathrm{M} = \mathrm{U}_{d'}\mathrm{S}_{d'}\mathrm{V}_{d'}{}^{\top}$$

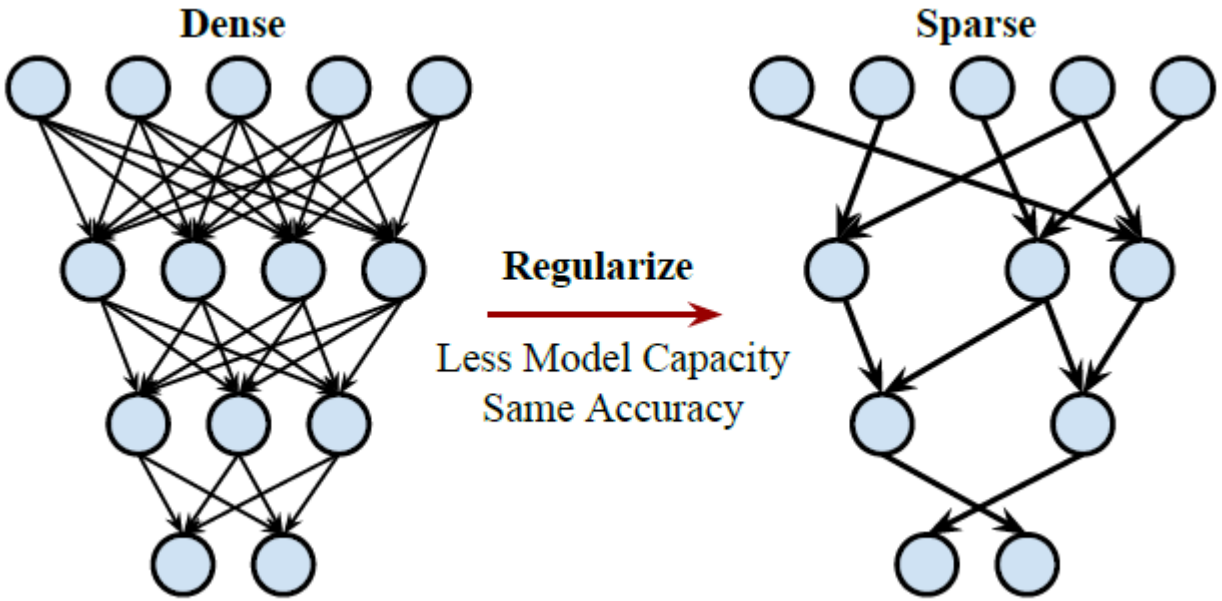
$$\mathrm{P} = \mathrm{U}_{d'}\mathrm{S}_{d'}^{1/2} \text{ and } \mathrm{Q} = \mathrm{V}_{d'}\mathrm{S}_{d'}^{1/2}$$

$$\min_{\mathrm{M}} \sum_i \|(\mathbf{y}_i - \bar{\mathbf{y}}) - \mathrm{M}(\mathbf{y}_i - \bar{\mathbf{y}})\|_2^2,$$

$$s.t. \quad rank(\mathrm{M}) \leq d'.$$

模型压缩与加速方法

剪枝（Pruning）



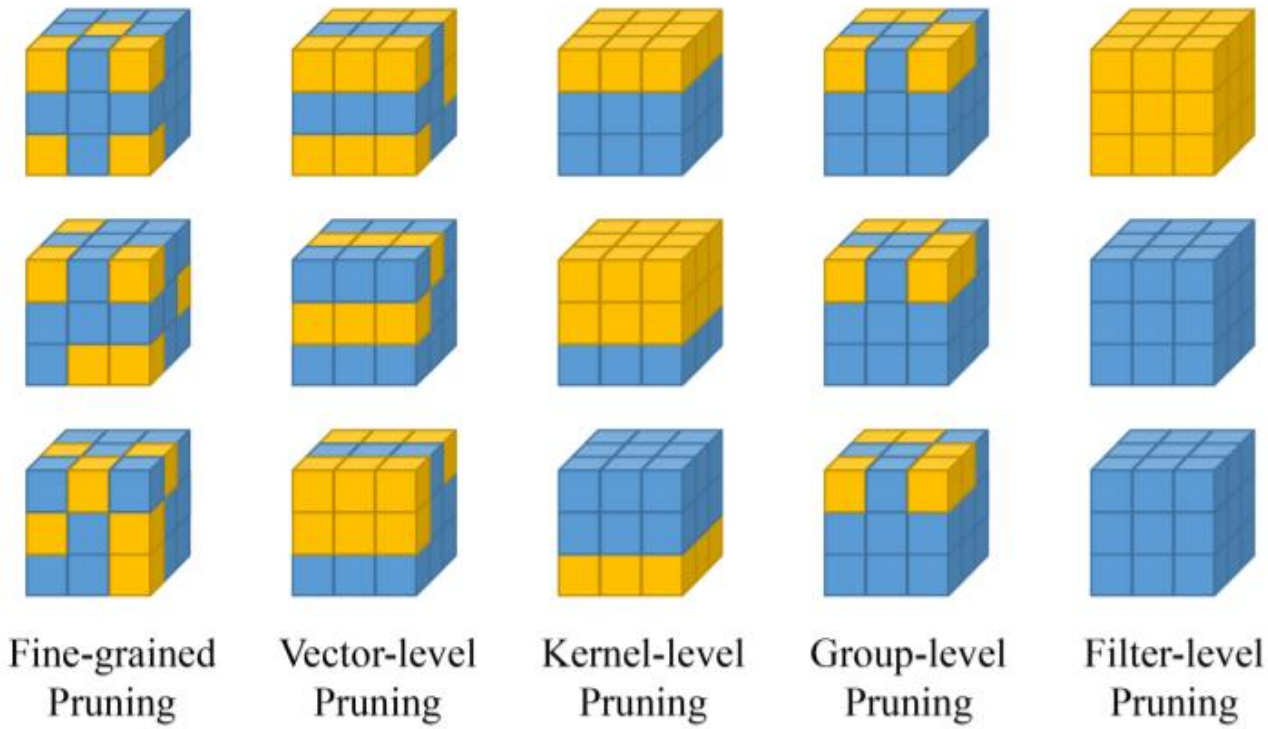
网络训练完成后，裁剪掉贡献信息量不大的网络参数。如果权值越少，就认为经过激活之后所产生的影响几乎可以忽略不计。去除权重比较小的连接，使得网络稀疏

剪枝的基本流程：

1. 衡量神经元的重要程度
2. 移除一部分不重要的神经元
3. 对网络进行微调
4. 返回第一步，进行下一轮剪枝

剪枝（Pruning）

剪枝（Pruning）



剪枝类型主要有：

- 细粒度剪枝
- 向量级别剪枝
- 核级别剪枝
- 组级别剪枝
- 滤波器级别剪枝

剪枝（Pruning）

剪枝方法分类

- 权重衰减法

  通过在网络目标函数中引入表示结构复杂性的正则化项来使训练的网络权重趋向稀疏化。缺点是正则化参数/权重剪枝阈值对剪枝结果影响很大

- 灵敏度量化法

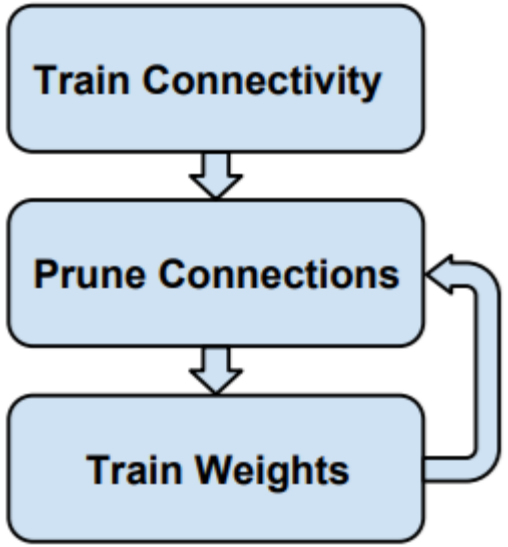
  通过设计一个灵敏度评价指标，衡量网络节点对于网络误差的贡献（灵敏度），删除那些贡献不大的节点。

- 根据相关性剪枝

  根据网络节点间的相关性或者相互作用进行剪枝，然后合并相关性较大的节点。类似于PCA一样，去除网络中信息冗余的网络节点。

模型压缩与加速概述

权重衰减法



L1正则化： $C = C_0 + \frac{\lambda}{n}\sum_w |w|.$

L2正则化： $C = C_0 + \frac{\lambda}{2n}\sum_w w^2,$

基本流程：

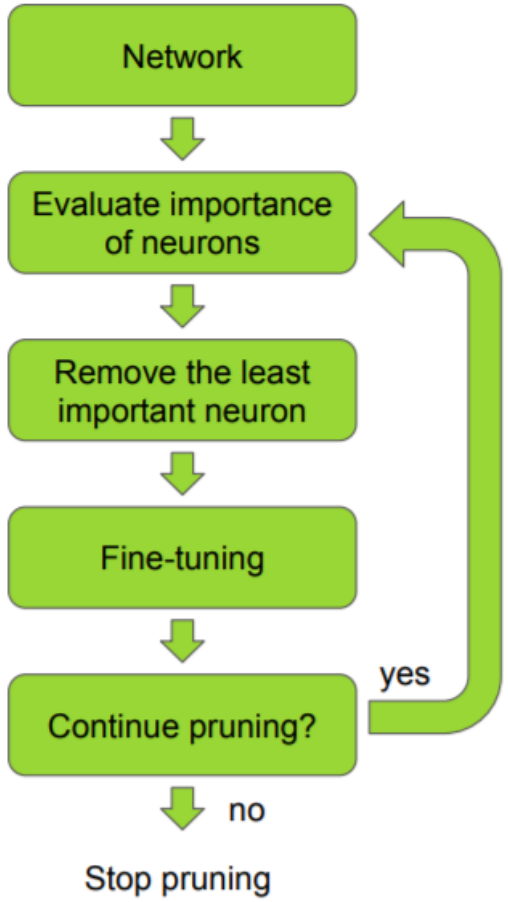
1. 在网络损失函数中加入正则化项保证网络稀疏
2. 设置一个阈值，删除网络权重低于阈值的网络节点
3. 对网络进行训练
4. 返回第一步，进行下一轮剪枝

缺点：

1. 传统方法需要在多个待测试阈值上重复迭代
2. 权重阈值在所有层共享，难以寻找合适的阈值
3. 对硬件支持并不友好

剪枝（Pruning）

灵敏度量化法



1. 对于网络节点h，我们假设其被裁剪时(h=0)，对网络损失的影响
2. 在h=0处进行泰勒展开

$$\mathcal{C}(\mathcal{D}|\mathcal{W},h=0) = \mathcal{C}(\mathcal{D},\mathcal{W}) - \frac{\delta\mathcal{C}}{\delta h}h + R_1(h=0)$$

3. 得到网络节点h对于网络损失的灵敏度表达

$$\left|\mathcal{C}(\mathcal{D}|\mathcal{W},h=0) - \mathcal{C}(\mathcal{D}|\mathcal{W})\right| = \left|\mathcal{C}(\mathcal{D}|\mathcal{W}) - \frac{\delta\mathcal{C}}{\delta h}h - \mathcal{C}(\mathcal{D}|\mathcal{W})\right| = \left|\frac{\delta\mathcal{C}}{\delta h}h\right|.$$

4. 迭代删除重要度最低的filter，并进行fine tune

剪枝（Pruning）

根据相关性剪枝

第 i 个隐层节点和第 j 个隐层节点的相关度公式为：

$$R_{ij}=\frac{\sum_p v_{ip}v_{jp}-pv_iv_j}{\left(\sum_p(v_{ip}-\overline{v_i})^2\right)^{\frac{1}{2}}\left(\sum_p(v_{jp}-\overline{v_j})^2\right)^{\frac{1}{2}}}$$

各节点的输出的方差公式如下：

$$s_i^2=\sum_p(v_{ip}-\overline{v_i})^2$$

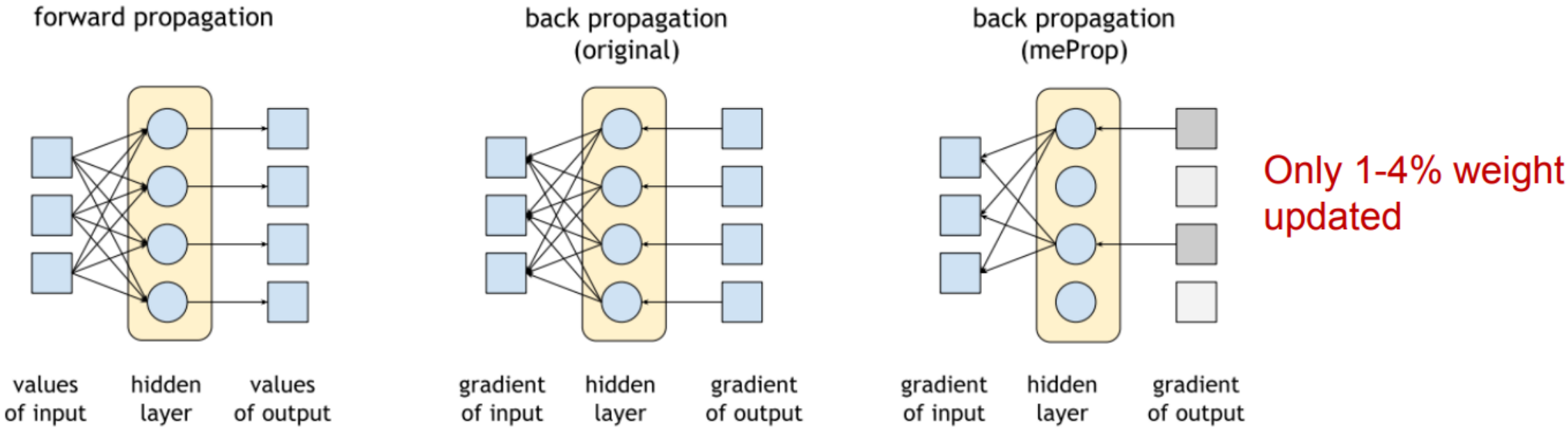
- 按以上公式计算后输出序列呈以下 5 种情况：
  1. 两个隐节点输出序列高度正相关；→ 合并
  2. 两个隐节点输出序列高度负相关；→ 合并
  3. 两个隐节点输出序列相关性不高；
  4. 某隐节点输出序列方差较小；→ 近似为常数
  5. 某隐节点输出序列方差较大。

- 可以处理的是线性相关度高的以及方差较小的隐层 节点。

剪枝（Pruning）

其他一些方法

- Keep top-k gradients of neural nodes , and prune rest of them
- Since some neural nodes' gradients are zeros, back propagation can be accelerated.

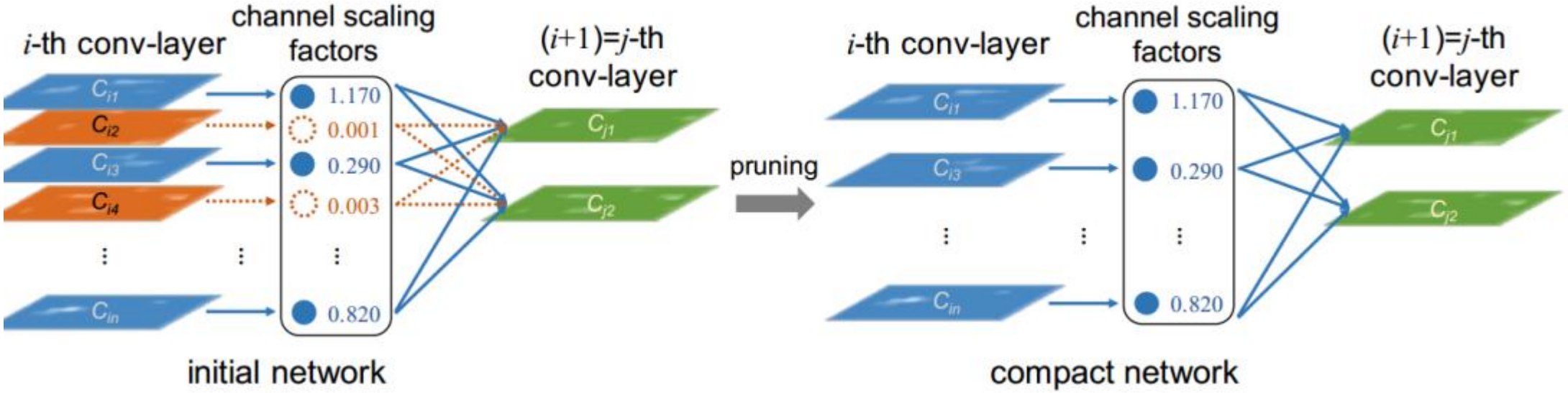


对梯度进行剪枝，每次只更新梯度大的神经元

剪枝（Pruning）

其他一些方法

- Associate a scaling factor with each channel
- Impose sparsity regularization on these scaling factors
- Prune those channels with small scaling factors



通过设计一个尺度因子，实现滤channel维度上的剪枝

模型压缩与加速概述

量化（Quantization）

一般而言，神经网络模型的参数都是用的32bit长度的浮点型数表示，实际上不需要保留那么高的精度，可以通过量化，比如用0~255表示原来32个bit所表示的精度，通过牺牲精度来降低每一个权值所需要占用的空间。此外，SGD（Stochastic Gradient Descent）所需要的精度仅为6~8bit，因此合理的量化网络也可保证精度的情况下减小模型的存储体积。

- 二值量化（+1，-1）
- 三值量化（+1，-1，0）
- 多值量化（8bit，16bit等）

量化的对象有：

- 网络权重
- 网络特征
- 网络梯度

量化(Quantization)

二值神经网络（Binary-Weight-Network）

- 基于符号函数Sign的确定性(deterministic)方法：

$$x^b = \text{Sign}(x) = \begin{cases} +1 & \text{if } x \geq 0, \\ -1 & \text{otherwise.} \end{cases}$$

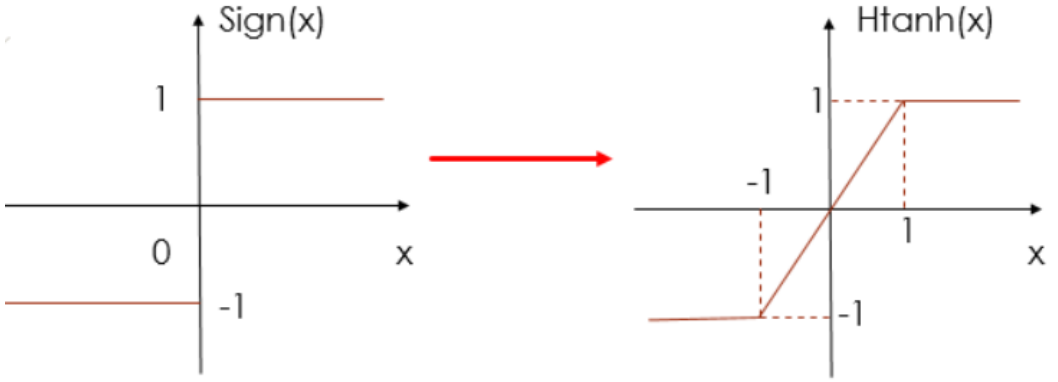
- 基于概率分布的随机二值化(stochastic)方法：

$$x^b = \begin{cases} +1 & \text{with probability } p = \sigma(x), \\ -1 & \text{with probability } 1 - p, \end{cases}$$

$$\sigma(x) = \text{clip}(\frac{x+1}{2}, 0, 1) = \max(0, \min(1, \frac{x+1}{2}))$$

- 对符号函数进行松弛求解来得到梯度：

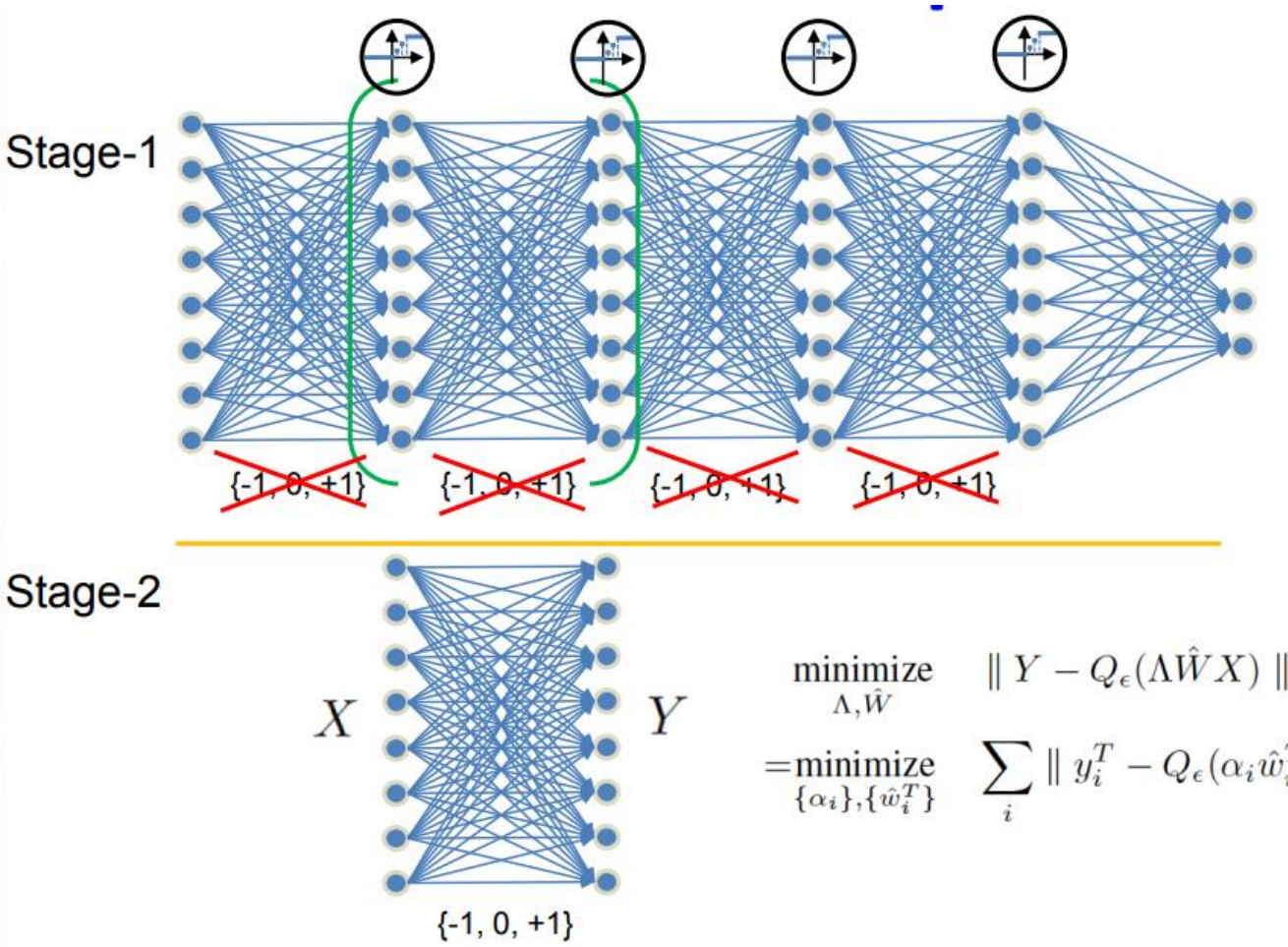
$$\text{Htanh}(x) = \text{Clip}(x, -1, 1) = \max(-1, \min(1, x)).$$



Courbariaux M, Hubara I, Soudry D, et al. Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1[J]. arXiv preprint arXiv:1602.02830, 2016.

量化(Quantization)

三值量化——示例



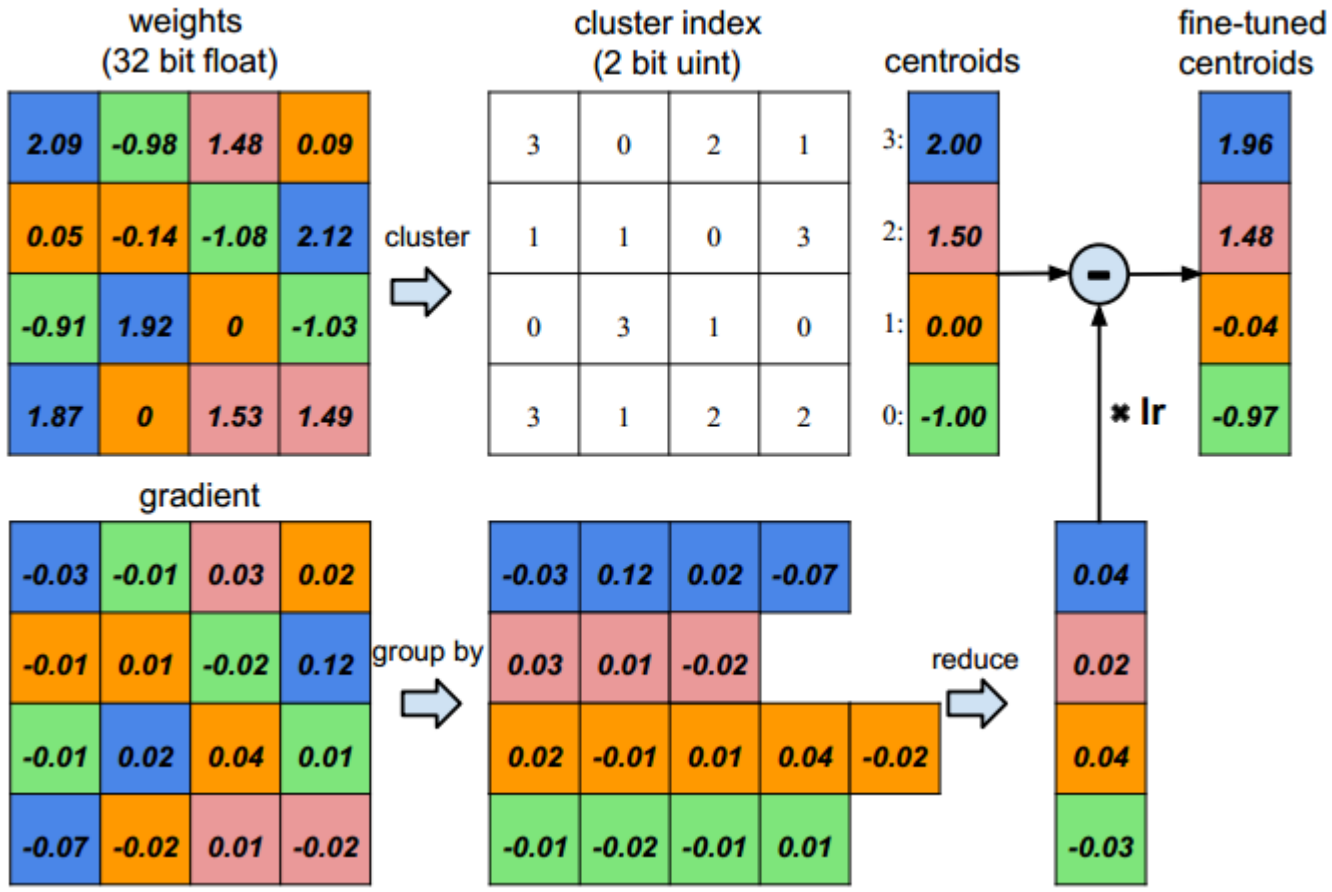
$$\underset{\Lambda,\hat{W}}{\text{minimize}} \quad \| Y - Q_\epsilon(\Lambda \hat{W} X) \|_F^2$$

$$= \underset{\{\alpha_i\},\{\hat{w}_i^T\}}{\text{minimize}} \quad \sum_i \| y_i^T - Q_\epsilon(\alpha_i \hat{w}_i^T X) \|_2^2$$

- Quantize both weight and actition is diffifult
- Inspired by Two-Step Hashing, Two-Step quantization method first quantizes the activation, then quantize the weights

量化(Quantization)

其他方法



- 先对权重、梯度进行聚类
- 根据聚类结果生成索引
- 根据索引得到结果

量化(Quantization)

网络量化对比

Method	Quantization			Acceleration	
	Weight	Activation	Gradient	Training	Testing
BinaryConnect [10]	Binary	Full	Full	No	Yes
BWN [65]	Binary	Full	Full	No	Yes
BWNH [32]	Binary	Full	Full	No	Yes
TWN [48]	Binary	Full	Full	No	Yes
FFN [81]	Ternary	Full	Full	No	Yes
INQ [99]	Ternary-5bit	Full	Full	No	Yes
BNN [65]	Binary	Binary	Full	No	Yes
XNOR [65]	Binary	Binary	Full	No	Yes
HWGQ [4]	Binary	2bit	Full	No	Yes
DoReFa-Net [100]	Binary	1-4bit	6bit, 8bit, Full	Yes	Yes

模型压缩与加速方法

各种方法的特点

方法名称	描述	应用场景	方法细节
低秩分解	使用矩阵对参数进行分解估计	卷积层和全连接层	标准化的途径，很容易实施，支持从零训练和预训练
剪枝	删除对准确率影响不大的参数	卷积层和全连接层	对不同设置具有鲁棒性，可以达到较好效果，支持从零训练和预训练
量化	减少表示每个权重所需的比特数来压缩原始网络	卷积层和全连接层	多依赖于二进制编码方式，适合在FPGA、单片机等平台上部署
知识蒸馏	训练一个更紧凑的神经网络来从大的模型蒸馏知识	卷积层和全连接层	模型表现对应用程序和网络结构较为敏感，只能从零开始训练
转移、紧凑卷积核	设计特别的卷积核来保存参数	只有卷积层	算法依赖于应用程序，通常可以取得好的表现，只能从零开始训练

End

